
FULLSTACK INTERVIEW PREP

Fullstack Interview Study Guide

React · Next.js · Node · TypeScript

A reorganized, fact-checked rebuild of the original guide. Tables no longer shatter across cells, code no longer clips at the right edge, missing answers are restored, and questions are grouped by topic with per-section numbering.

Sections	9
Questions	125
Format	A4, study-card style
Edition	Rebuilt June 2026

Contents

1. Frontend Fundamentals — 8 questions

- 1.01** Cookie Storage, Local Storage, and Session Storage
- 1.02** Optimizations With Bundlers (Webpack, Vite, ...)
- 1.03** XSS — Cross-Site Scripting
- 1.04** Micro Frontends
- 1.05** FCP — First Contentful Paint
- 1.06** Monitoring Frontend Apps — Tools & Metrics
- 1.07** Testing Frontend Apps — Unit, Integration, E2E
- 1.08** Deploying a Frontend App (React or Next.js)

2. CSS — 11 questions

- 2.01** Difference Between em and rem
- 2.02** The CSS Box Model
- 2.03** Selectors Specificity
- 2.04** Flexbox and Grid
- 2.05** Media Queries
- 2.06** Pseudo-Classes and Pseudo-Elements
- 2.07** CSS-in-JS
- 2.08** Sass and Less
- 2.09** position Property
- 2.10** Viewport Units (vh, vw, dvh, ...)
- 2.11** z-index

3. JavaScript — 19 questions

- 3.01 What is an IIFE?
- 3.02 What is a Closure?
- 3.03 var, let, and const
- 3.04 this in JavaScript
- 3.05 Hoisting
- 3.06 The Event Loop in JavaScript
- 3.07 Prototype
- 3.08 Promise.all vs Promise.race vs Promise.allSettled vs Promise.any
- 3.09 Callbacks
- 3.10 Arrow Functions
- 3.11 The Module Pattern
- 3.12 Currying
- 3.13 Tail Recursion
- 3.14 Garbage Collection and Memory Management
- 3.15 Generator Functions
- 3.16 Promises
- 3.17 OOP in JavaScript
- 3.18 Shallow Copy vs Deep Copy
- 3.19 Synchronous vs Asynchronous Functions

4. TypeScript — 12 questions

- 4.01 What is as const?
- 4.02 Private Access Modifiers
- 4.03 Type Guards
- 4.04 Structural vs Nominal Typing
- 4.05 What is void and when to use it?
- 4.06 unknown vs never
- 4.07 Union Types (the | operator)
- 4.08 Mixins in TypeScript
- 4.09 Type Assertions
- 4.10 Template Literal Types
- 4.11 Decorators
- 4.12 Parameter Properties (constructor shorthand)

5. React — 25 questions

- 5.01 What is JSX?
- 5.02 Functional vs Class Components
- 5.03 Props vs State
- 5.04 Controlled vs Uncontrolled Components
- 5.05 The key attribute in lists
- 5.06 Virtual DOM
- 5.07 Lifecycle Methods (Class Components)
- 5.08 React Hooks — overview
- 5.09 How Hooks Work Under the Hood
- 5.10 Context API
- 5.11 Higher-Order Components (HOCs)
- 5.12 Reconciliation
- 5.13 React Portals
- 5.14 React Strict Mode
- 5.15 The Spread Operator in React
- 5.16 Conditional Rendering
- 5.17 useEffect Dependency Array
- 5.18 useReducer
- 5.19 React.lazy and Suspense
- 5.20 Optimizing Performance in React Apps
- 5.21 Reactive Programming in React
- 5.22 Passing Data from Child to Parent
- 5.23 Why can't useEffect be async?
- 5.24 React Fiber vs the Virtual DOM
- 5.25 How React Decides When to Re-render

6. Redux — 9 questions

- 6.01 What is Redux and How is it Structured?
- 6.02 Data Flow in React + Redux
- 6.03 The Role of the Store
- 6.04 The Role of a Reducer
- 6.05 Local Component State vs Redux State
- 6.06 Provider — How Components Get State
- 6.07 connect() — the Pre-Hooks API
- 6.08 Redux Middleware
- 6.09 Redux Thunk

7. Next.js — 14 questions

- 7.01 What is Next.js?
- 7.02 SSR vs SSG
- 7.03 File-Based Routing
- 7.04 getStaticProps (Pages Router / SSG)
- 7.05 getServerSideProps (Pages Router / SSR)
- 7.06 getStaticPaths
- 7.07 Dynamic Routes
- 7.08 Image Optimization (next/image)
- 7.09 Client-Side Routing (next/link)
- 7.10 React Server Components (App Router)
- 7.11 SEO in Next.js
- 7.12 Route Protection (Auth)
- 7.13 ISR — Incremental Static Regeneration
- 7.14 Middleware in Next.js

8. Node.js — 13 questions

- 8.01 What is Node.js?
- 8.02 How Node.js Works Under the Hood
- 8.03 Why is Node.js Single-Threaded?
- 8.04 How Node Handles Concurrency
- 8.05 Modules in Node.js (CommonJS vs ES Modules)
- 8.06 V8 Engine
- 8.07 Control Flow in Node.js
- 8.08 CORS — Cross-Origin Resource Sharing
- 8.09 Streams and Piping
- 8.10 The Cluster Module
- 8.11 WebSockets in Node.js
- 8.12 package.json
- 8.13 npm — Node Package Manager

9. Backend & APIs — 14 questions

- 9.01 RESTful APIs
- 9.02 GraphQL vs REST
- 9.03 SOLID Principles
- 9.04 Microservices
- 9.05 Payload, Request, and Response
- 9.06 Anatomy of an HTTP Request
- 9.07 REST API Caching
- 9.08 REST API Best Practices
- 9.09 SQL Injection
- 9.10 HTTP Caching
- 9.11 Scaling a SQL Database
- 9.12 Deployment Strategies
- 9.13 JWT — JSON Web Token
- 9.14 SQL vs NoSQL Databases

SECTION 01

Frontend Fundamentals

8 questions

Q 1.01

Cookie Storage, Local Storage, and Session Storage

Feature	Cookie	localStorage	sessionStorage
Capacity	~4 KB	5–10 MB	5–10 MB
Sent to server	Yes, on every request	No	No
Expiration	Configurable; default session	Until manually cleared	On tab/window close
Scope	Per domain & path	Per origin, shared across tabs	Per tab and per origin
Typical use	Auth tokens, server-readable state	App state, user preferences	One-off form data, per-tab workflow state

Security note: set `HttpOnly`, `Secure`, and `SameSite` on cookies that carry auth. `localStorage` is accessible to any JS on the page — never store tokens there if you can avoid it.

Q 1.02

Optimizations With Bundlers (Webpack, Vite, ...)

- **Bundler** — Tools like Webpack, Vite, or Rollup take your JS, CSS, images and assets and combine them into optimized files served to the browser.
- **Polyfill** — JavaScript that adds missing functionality to older browsers (e.g. `Promise` in IE11). Often combined with feature detection so modern browsers don't pay the cost.
- **Tree shaking** — Removes unused exports from the final bundle. Requires ES modules (`import` / `export`) and `"sideEffects": false` hints in `package.json` for libraries.
- **Code splitting** — Splits the bundle by route or dynamic import, so the browser only downloads what's needed for the current view.
- **Lazy loading** — Defers loading non-critical modules until they're needed (React `lazy` + `Suspense`, dynamic `import()`).
- **Minification** — Removes whitespace, shortens identifiers, drops dead code via Terser/esbuild.
- **Source maps** — Map minified code back to the original for debugging in production.
- **Asset optimization** — Compress images (AVIF/WebP), inline small SVGs, use content-hashed filenames for long-term caching.

Vite vs. Webpack: Vite uses native ES modules + esbuild in dev for fast HMR, then Rollup for production. Webpack bundles everything from dev start, which is slower on large projects but is more configurable and battle-tested.

Q 1.03

XSS — Cross-Site Scripting

XSS happens when an attacker injects malicious script into pages viewed by other users. The script runs in the victim's browser with the page's origin, so it can read cookies, hijack sessions, exfiltrate data, or perform actions on the user's behalf.

THREE TYPES

- **Stored** — Payload saved on the server (comment, profile bio) and served to every viewer.
- **Reflected** — Payload comes in via URL/form params and is reflected back in the response without sanitization.
- **DOM-based** — Vulnerability is in client-side code that writes untrusted data to the DOM (`innerHTML`, `document.write`).

DEFENSES

- Treat all user input as untrusted. Escape on output for the destination context (HTML, attribute, JS, URL, CSS).
- Prefer frameworks that auto-escape (React's JSX, Vue templates).
- Avoid `dangerouslySetInnerHTML` / `v-html`; if unavoidable, sanitize with DOMPurify.
- Set a strict **Content Security Policy** — `script-src 'self'` blocks inline and remote scripts not whitelisted.
- Set cookies as `HttpOnly` so JS can't read auth tokens.

Q 1.04

Micro Frontends

Micro frontends apply the microservices idea to the UI. Instead of one monolithic SPA, the UI is split into smaller, independently deployable apps — each owned by a different team — that compose into one product at runtime or build time.

WHEN IT HELPS

- Multiple teams shipping at different cadences on the same product.
- Different parts of the app have very different tech needs and lifecycles.
- Gradual migration off a legacy frontend.

COMPOSITION STRATEGIES

- **Build-time** — npm packages per micro frontend, composed in a shell app. Tight coupling, simple deploys.
- **Runtime via iframes** — strongest isolation; weak UX (navigation, shared state).
- **Module Federation (Webpack 5 / Vite Federation)** — runtime composition with shared deps. Most common modern approach.

- **Web Components** — framework-agnostic custom elements as the integration contract.

Trade-offs: independent deploys and team autonomy come at the cost of bundle duplication, harder cross-app state, design system fragmentation, and operational complexity. Don't reach for them on a small team.

Q 1.05 FCP — First Contentful Paint

FCP is a Core Web Vitals metric that measures the time from when the page starts loading to when any text, image, SVG, or non-blank canvas is rendered.

THRESHOLDS (GOOGLE)

- **Good:** ≤ 1.8 s
- **Needs improvement:** 1.8 s – 3.0 s
- **Poor:** > 3.0 s

HOW TO IMPROVE IT

- Reduce server response time (CDN, edge caching, faster origin).
- Eliminate render-blocking resources (defer non-critical JS/CSS, inline critical CSS).
- Preload key fonts; use `font-display: swap`.
- Avoid huge above-the-fold JS; ship server-rendered HTML for first paint.

Related metrics: LCP (Largest Contentful Paint) — when the biggest element is painted; TTI — when the page becomes interactive; CLS — visual stability; INP — responsiveness to interactions.

Q 1.06 Monitoring Frontend Apps — Tools & Metrics

WHAT TO MONITOR

- **Performance:** FCP, LCP, TTI, TBT, CLS, INP, page load time.
- **Errors:** uncaught exceptions, unhandled promise rejections, failed API calls, breadcrumb trails.
- **UX / engagement:** interaction delay, bounce rate, rage clicks, conversion funnels.
- **Network & resources:** API response times, asset sizes, cache hit ratios, failed requests.

TOOLS

- **RUM (Real User Monitoring):** Sentry, Datadog RUM, LogRocket, New Relic Browser, Vercel Analytics.
- **Synthetic / lab:** Lighthouse, WebPageTest, Pingdom, Google PageSpeed Insights.

- **Bundle analysis:** webpack-bundle-analyzer, source-map-explorer, Vite's `--report`.

Best practice: track real user metrics (RUM), not just synthetic. Alert on CLS, LCP, and INP regressions. Combine performance and error data on one dashboard. Keep monitoring on after deploys — especially after big releases.

Q 1.07

Testing Frontend Apps — Unit, Integration, E2E

Type	What it tests	Tools
Unit	One function or component in isolation; mock everything external.	Jest, Vitest, React Testing Library (for components)
Integration	Several units working together — e.g. a form component with its validation hook and store.	React Testing Library, MSW (mock HTTP)
E2E	Whole app in a real browser; simulates a user flow end-to-end.	Cypress, Playwright
Visual regression	Catches unintended UI changes.	Chromatic, Percy, Playwright snapshots

Test pyramid: lots of fast unit tests, fewer integration tests, a small number of E2E tests covering critical flows. RTL's philosophy is to test behavior the user sees, not implementation details — avoid asserting on internal state or rendered classnames.

Q 1.08

Deploying a Frontend App (React or Next.js)

REACT SPA (VITE / CRA / SIMILAR)

- **Step 1 — Build:** `npm run build` produces a `dist/` or `build/` folder with static HTML, JS, and CSS.
- **Step 2 — Choose a host:** Vercel, Netlify, Cloudflare Pages, GitHub Pages, Firebase Hosting, AWS S3 + CloudFront.
- **Step 3 — Configure routing:** SPAs need a fallback to `index.html` for client-side routes (rewrite rule on the host).
- **Step 4 — Cache headers:** long-cache content-hashed assets, no-cache for `index.html`.

NEXT.JS

- **SSG (static):** can deploy to any static host (Vercel, Netlify, S3) like an SPA.

- **SSR / API routes:** needs a Node runtime — Vercel, AWS Amplify, Render, Fly.io, or self-hosted Node + reverse proxy.
- **ISR:** supported natively on Vercel; on other hosts use the standalone output and configure caching.

Host	Best for
Vercel	Next.js (by its creators), SSR/SSG/ISR out of the box.
Netlify	Static sites and SPAs, good DX.
Cloudflare Pages	Static + edge functions, generous free tier.
AWS S3 + CloudFront	Enterprise static hosting at scale.
AWS / DigitalOcean / Render	Custom Node server hosting for SSR apps.

SECTION 02

CSS

11 questions

Q 2.01

Difference Between em and rem

- **em** — relative to the *parent element's* font-size. Compounds when nested.
- **rem** — relative to the *root* `<html>` font-size (usually 16px unless overridden). Doesn't compound.

Rule of thumb: use `rem` for global sizing (typography, spacing, layout) so things stay consistent and users can scale the whole UI with browser zoom. Use `em` for component-local sizing where you want children to scale with the container's font-size (e.g. a button's padding that matches its text).

Q 2.02

The CSS Box Model

Every HTML element is rendered as a rectangular box. The box model defines how the total size is calculated from four parts:

- **Content** — the actual text, image, or media inside.
- **Padding** — space between content and border (transparent; takes background color).
- **Border** — surrounds the padding.
- **Margin** — space outside the border (transparent; collapses vertically between adjacent block elements).

BOX-SIZING

- `content-box` (default) — `width` applies to content only; padding and border are added on top.
- `border-box` — `width` includes content + padding + border. Almost always what you want.

```
* { box-sizing: border-box; } /* the standard reset */
```

Q 2.03

Selectors Specificity

When multiple rules target the same element, the browser uses **specificity** to decide which wins. Specificity is a four-tuple `(a, b, c, d)` compared left-to-right:

Position	Counts	Example
a	Inline styles	<code>style="color: red"</code>
b	IDs	<code>#header</code>
c	Classes, attributes, pseudo-classes	<code>.btn</code> , <code>[type="text"]</code> , <code>:hover</code>
d	Elements and pseudo-elements	<code>div</code> , <code>p</code> , <code>::before</code>

`!important` beats everything in the cascade except a more important rule, and is best avoided. The universal selector `*` and combinators (`>`, `+`, `~`) add no specificity.

EXAMPLE

- `p` → (0,0,0,1)
- `.btn` → (0,0,1,0) — beats `p`
- `#nav .btn` → (0,1,1,0) — beats `.btn`
- `style="..."` → (1,0,0,0) — beats `#nav .btn`

Prefer class-based selectors for maintainability. Reach for IDs sparingly and avoid `!important` as anything other than a last resort.

Q 2.04 Flexbox and Grid

Feature	Flexbox	Grid
Dimensions	One-dimensional (row OR column)	Two-dimensional (rows AND columns)
Best for	Aligning items within a single line: navbars, toolbars, card rows.	Page-level layouts: dashboards, galleries, complex forms.
Axes	Main axis + cross axis	Rows and columns; explicit and implicit tracks
Item placement	Order via <code>flex-grow</code> / <code>shrink</code> / <code>basis</code>	Explicit placement via <code>grid-column</code> / <code>grid-row</code> / <code>grid-area</code>

FLEXBOX — ONE-DIMENSIONAL

```
.container {
  display: flex;
  flex-direction: row;          /* or column */
  justify-content: space-between; /* main-axis alignment */
  align-items: center;          /* cross-axis alignment */
  gap: 1rem;
}
.item { flex: 1; }              /* equal-width items */
```

GRID — TWO-DIMENSIONAL

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: auto;
  gap: 1rem;
}
```



```

}
.item:nth-child(1) {
  grid-column: 1 / 3;          /* span columns 1-2 */
}

```

They compose well: use Grid for the page skeleton, Flexbox inside grid cells for fine-grained alignment.

Q 2.05 Media Queries

Media queries apply CSS conditionally based on the device, screen size, or user preferences (dark mode, reduced motion).

Device	Typical width	Query
Small phones	≤ 480px	<code>@media (max-width: 480px) { ... }</code>
Tablets	≤ 768px	<code>@media (max-width: 768px) { ... }</code>
Small laptops	≤ 1024px	<code>@media (max-width: 1024px) { ... }</code>
Desktops	≥ 1200px	<code>@media (min-width: 1200px) { ... }</code>

BEYOND VIEWPORT SIZE

```

@media (prefers-color-scheme: dark) { ... }
@media (prefers-reduced-motion: reduce) { ... }
@media (hover: hover) { ... } /* devices with a real hover */
@media (orientation: landscape) { ... }

```

Mobile-first is the modern default: write base styles for the smallest screen, then progressively enhance with `min-width` queries.

Q 2.06 Pseudo-Classes and Pseudo-Elements

Feature	Pseudo-class	Pseudo-element
Targets	An element's state or position	A specific part of an element, or inserted content
Syntax	Single colon (<code>:</code>)	Double colon (<code>::</code>) in modern CSS
Examples	<code>:hover</code> , <code>:focus</code> , <code>:first-child</code> , <code>:checked</code> , <code>:not()</code>	<code>::before</code> , <code>::after</code> , <code>::first-line</code> , <code>::placeholder</code> , <code>::selection</code>

```

a:hover { color: red; }           /* state */
li:first-child { font-weight: bold; } /* position */

p::first-line { font-variant: small-caps; } /* part of element */
.button::before { content: "▶ "; } /* inserted content */

```

`::before` and `::after` require a `content` property — even `content: ""` — to actually render.

Q 2.07

CSS-in-JS

CSS-in-JS means writing styles inside JavaScript (or TypeScript) files instead of separate `.css` files. Styles are typically defined as JS objects or tagged template literals and scoped to a single component.

WHY USE IT

- **Scoped styles** — no global namespace conflicts.
- **Dynamic styling** — styles can read from props or state.
- **Theming** — light/dark mode and design tokens propagate via context.
- **Co-location** — styles live next to the component that owns them.

Library	Style	Notes
styled-components	Tagged template literals	Runtime; popular but adds bundle weight.
Emotion	Template literals or object styles	Similar to styled-components, often faster.
JSS	Object-based	Used by older Material-UI.
vanilla-extract	TS, zero-runtime	Compiles to static CSS at build time.
Tailwind CSS	Utility classes	Not strictly CSS-in-JS, but a popular alternative — atomic classes co-located in JSX.

Trade-off: runtime CSS-in-JS has a real performance cost (JS parse + style generation). Modern projects often prefer zero-runtime (vanilla-extract, Linaria) or utility-first (Tailwind) approaches.

Q 2.08

Sass and Less

Both are **CSS preprocessors**: they extend CSS with variables, nesting, mixins, and functions, then compile to standard CSS the browser can read.

Feature	Sass / SCSS	Less
Variables	<code>\$color: red;</code>	<code>@color: red;</code>
Nesting	Yes	Yes
Mixins	<code>@mixin</code> / <code>@include</code>	<code>.mixin()</code>
Functions	<code>lighten()</code> , <code>darken()</code> , <code>map-get()</code> , custom	<code>lighten()</code> , <code>darken()</code>
Imports	<code>@use</code> , <code>@forward</code> , <code>@import</code> (deprecated)	<code>@import</code>
Extensions	<code>.scss</code> or <code>.sass</code>	<code>.less</code>

Native CSS now supports variables (custom properties) and nesting, so many projects skip preprocessors entirely. Sass still wins for advanced features like maps, loops, and mixins.

Q 2.09

position Property

Value	Behavior	Uses top/left/etc.?
<code>static</code>	Default. Normal document flow.	No
<code>relative</code>	Positioned relative to its own original spot. Still takes space in the flow.	Yes
<code>absolute</code>	Removed from the flow. Positioned relative to the nearest <i>positioned</i> ancestor (or <code><body></code>).	Yes
<code>fixed</code>	Removed from the flow. Positioned relative to the viewport.	Yes
<code>sticky</code>	Acts like <code>relative</code> until the scroll threshold is reached, then "sticks" like <code>fixed</code> .	Yes

```
.parent { position: relative; }      /* anchor for absolute children */
.child  { position: absolute; top: 0; right: 0; }
.header { position: fixed; top: 0; width: 100%; }
nav     { position: sticky; top: 0; }
```

`sticky` only works while the element is inside the scrolling container; if any ancestor has `overflow: hidden` or `overflow: auto`, that becomes the scroll context.

Q 2.10 Viewport Units (vh, vw, dvh, ...)

`vh` represents 1% of the viewport's height; `100vh` is the full viewport height. `vw` works the same way for width.

Feature	vh / vw (viewport)	% (parent)
Reference	The visible browser window	The parent element's size
Needs a sized parent?	No	Yes — parent must have a defined height/width
Reacts to window resize	Always	Only if the parent does
Typical use	Hero sections, full-screen modals	Filling a container

MOBILE VIEWPORT CAVEAT

Mobile browsers count `100vh` with the URL bar hidden, so `height: 100vh` overflows when the bar is visible. Modern CSS adds:

- `svh` / `svw` — *small* viewport (UI showing)
- `lvh` / `lvw` — *large* viewport (UI hidden)
- `dvh` / `dvw` — *dynamic* viewport (changes as UI shows/hides)

```
.hero { height: 100dvh; } /* the modern fix for mobile */
```

Q 2.11 z-index

`z-index` controls the stacking order of overlapping elements on the z-axis (front-to-back). Higher values appear on top.

KEY RULES

- Only works on *positioned* elements (`relative`, `absolute`, `fixed`, `sticky`) or on flex/grid items.
- Default is `auto` — elements stack in document order.
- Negative values are allowed.

STACKING CONTEXTS

A stacking context is a self-contained "world" of z-index. Inside it, children stack relative to each other; relative to outside elements, the whole context stacks as one. A new stacking context is created by:

- `position` other than `static` with any `z-index` (even `0`).
- `opacity` less than 1.
- `transform`, `filter`, `perspective`, `will-change`.
- `isolation: isolate`.

If `z-index: 9999` "isn't working", it's almost always a stacking-context issue — the element is trapped inside a context that itself sits lower in the global stack.

SECTION 03

JavaScript

19 questions

Q 3.01

What is an IIFE?

An **Immediately Invoked Function Expression** is a function that runs as soon as it's defined. Useful (historically) for creating a private scope before `let` / `const` and ES modules existed.

```
(function () {  
  const secret = 42;  
  console.log("Runs immediately");  
})();  
  
// Arrow form  
(() => console.log("Same idea"))();
```

Modern code rarely needs IIFEs — `let` / `const` give block scope, and ES modules give module scope. Still common in bundled UMD libraries and quick one-off scripts.

Q 3.02

What is a Closure?

A **closure** is a function that "remembers" the variables from the scope in which it was created, even after that outer scope has finished executing. In JS, every function is a closure over its enclosing lexical scope.

```
function makeCounter() {  
  let count = 0;           // captured by the inner function  
  return function () {  
    return ++count;  
  };  
}  
  
const counter = makeCounter();  
counter(); // 1  
counter(); // 2  
counter(); // 3
```

WHY IT MATTERS

- Data privacy / encapsulation (module pattern).
- Partial application and currying.
- Event handlers and callbacks that remember context.
- Memoization caches.

Q 3.03

var, let, and const

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes, initialized to <code>undefined</code>	Yes, but in the Temporal Dead Zone until declared	Yes, in TDZ until declared
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed (the binding; object contents can still mutate)
Use when	Legacy code only	Variable needs to change	Default — bindings that never change

```
function demo() {
  if (true) {
    var x = 10;
    let y = 20;
    const z = 30;
  }
  console.log(x); // 10 — var leaked out
  console.log(y); // ReferenceError — block-scoped
  console.log(z); // ReferenceError — block-scoped
}
```

`const obj = {}` doesn't make the object immutable — only the binding. Use `Object.freeze()` or `readonly` in TS for true immutability.

Q 3.04

this in JavaScript

`this` is a special binding determined by *how* a function is called, not where it's defined (except for arrow functions).

Call style	What <code>this</code> is
Method call (<code>obj.fn()</code>)	The object before the dot (<code>obj</code>)
Standalone function (<code>fn()</code>)	<code>undefined</code> in strict mode, <code>globalThis</code> otherwise

Call style	What this is
Constructor (<code>new Fn()</code>)	The newly created instance
Explicit (<code>fn.call(ctx)</code> / <code>.apply</code> / <code>.bind</code>)	Whatever you pass
Arrow function	Inherited from the surrounding lexical scope — never rebound
DOM event handler	The element that received the event (for traditional functions)

```
const obj = {
  name: "Dincho",
  greet: function () { return "Hi " + this.name; },      // "Hi Dincho"
  greetArrow: () => "Hi " + this.name,                  // arrow ignores obj
};

const fn = obj.greet;
fn(); // "Hi undefined" – lost context
```

Q 3.05 Hoisting

Hoisting is the engine moving declarations to the top of their scope during the compile phase. The behavior differs by declaration type:

- **var** — declaration hoisted and initialized to `undefined`. Reading before assignment yields `undefined`.
- **let** / **const** — declaration hoisted, but uninitialized. Reading before the declaration throws `ReferenceError` (the Temporal Dead Zone).
- **Function declarations** — fully hoisted, including the body. Can be called before they appear.
- **Function expressions** — only the variable is hoisted, not the function value.
- **Classes** — hoisted but in the TDZ; can't be used before the declaration.

```
console.log(a);    // undefined (var hoisted, no value yet)
var a = 1;

console.log(b);    // ReferenceError (TDZ)
let b = 2;

greet();           // works – function declaration fully hoisted
function greet() { console.log("hi"); }
```

```
sayHi(); // TypeError: sayHi is not a function
var sayHi = function () { console.log("hi"); };
```

Q 3.06 The Event Loop in JavaScript

JavaScript is single-threaded but achieves non-blocking I/O through the **event loop**. Async work runs outside the call stack and is scheduled back through task queues.

THE PIECES

- **Call stack** — the synchronous execution stack.
- **Web APIs / Node APIs** — timers, network, file I/O — handled by the host environment, not the JS engine.
- **Macrotask queue** — `setTimeout`, `setInterval`, `setImmediate` (Node), I/O callbacks, UI events.
- **Microtask queue** — Promise `then` / `catch` / `finally`, `queueMicrotask`, `MutationObserver`. Always drained *before* the next macrotask.

THE LOOP

1. Run the synchronous code until the stack is empty.
2. Drain the entire microtask queue.
3. Run one macrotask.
4. Repeat from step 2.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// 1, 4, 3, 2 - microtasks (Promise) beat macrotasks (setTimeout)
```

In Node, the loop has additional phases (timers, pending callbacks, poll, check, close). `process.nextTick` has its own queue drained before microtasks.

Q 3.07 Prototype

Every JavaScript object has an internal link to another object called its **prototype**. When you access a property, the engine walks up the **prototype chain** until it finds the property or reaches `null`.

```
const animal = { eats: true };
const dog = Object.create(animal);
```

```
dog.barks = true;

dog.barks; // true   - own property
dog.eats;  // true   - found on the prototype
dog.flies; // undefined - end of chain
```

HOW JS GETS "CLASSES"

```
function Person(name) { this.name = name; }
Person.prototype.greet = function () { return "Hi " + this.name; };

const p = new Person("Dincho");
p.greet(); // "Hi Dincho"

// ES6 class syntax is sugar for the same thing:
class Person2 {
  constructor(name) { this.name = name; }
  greet() { return "Hi " + this.name; }
}
```

Inspect with `Object.getPrototypeOf(obj)` ; modify (sparingly!) with `Object.setPrototypeOf(obj, proto)` .

Q 3.08

Promise.all vs Promise.race vs Promise.allSettled vs Promise.any

Combinator	Resolves with	Rejects when
<code>Promise.all</code>	An array of every resolved value, in input order.	Any input rejects — fails fast.
<code>Promise.allSettled</code>	An array of <code>{status, value}</code> or <code>{status, reason}</code> for every input.	Never rejects.
<code>Promise.race</code>	The first to settle (resolve OR reject).	The first input that rejects, if it wins the race.
<code>Promise.any</code>	The first one that <i>resolves</i> .	Only if all of them reject — then rejects with an <code>AggregateError</code> .

```
const a = fetch("/a");
const b = fetch("/b");
const c = fetch("/c");

await Promise.all([a, b, c]); // need all to succeed
await Promise.allSettled([a, b, c]); // want every outcome
```

```
await Promise.race([a, b, c]); // whichever finishes first
await Promise.any([a, b, c]); // first one that succeeds
```

Q 3.09 Callbacks

A **callback** is a function passed as an argument to another function, to be invoked later — either synchronously (like `Array.map`) or asynchronously (like `setTimeout`, `fs.readFile`).

```
// Synchronous callback
[1, 2, 3].map(x => x * 2); // map calls the callback for each element

// Asynchronous callback (Node-style)
fs.readFile("./data.txt", "utf8", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

CALLBACK HELL

Nesting many async callbacks makes code hard to read and reason about. Promises and `async/await` are the modern replacements:

```
// Promise-based
readFile("./data.txt", "utf8")
  .then(parse)
  .then(transform)
  .then(save)
  .catch(handleError);

// async/await
try {
  const raw = await readFile("./data.txt", "utf8");
  const data = parse(raw);
  await save(transform(data));
} catch (err) { handleError(err); }
```

Q 3.10 Arrow Functions

Arrow functions are a shorter syntax for function expressions, with two key differences from regular functions:

- **Lexical `this`** — they don't have their own `this`. They use whatever `this` is in the surrounding scope.
- **No `arguments` object** — use rest params (`...args`) instead.
- Can't be used with `new` (no `[[Construct]]`).

- Can't be generators (`function*`).

```
const square = x => x * x;
const add = (a, b) => a + b;
const obj = {
  count: 0,
  // Works – arrow inherits `this` from `start`, which is `obj`
  start() { setInterval(() => { this.count++; }, 1000); },
};
```

Don't use arrows for object methods that need `this` to refer to the object — they'll bind `this` to the surrounding scope (often `window` / `undefined`).

Q 3.11 The Module Pattern

A pre-ES-modules pattern that uses an IIFE plus a closure to create private state and expose a public API. It's the closure example, structured for module-like reuse.

```
const Counter = (function () {
  let count = 0; // private

  function reset() { count = 0; } // private

  return { // public API
    increment() { count++; },
    decrement() { count--; },
    value() { return count; },
    reset,
  };
})();

Counter.increment();
Counter.value(); // 1
Counter.count; // undefined – truly private
```

Modern code uses ES modules (`export` / `import`) for the same goals: anything not exported is module-private.

Q 3.12 Currying

Currying transforms a function that takes multiple arguments into a chain of functions that each take one argument.

```
// Normal
function add(a, b, c) { return a + b + c; }
add(1, 2, 3); // 6

// Curried
const addC = a => b => c => a + b + c;
addC(1)(2)(3); // 6

// Partial application
const add1 = addC(1);
const add1and2 = add1(2);
add1and2(10); // 13
```

WHY USE IT

- Partial application — fix some arguments now, supply the rest later.
- Compose small, reusable utilities (popular in functional libraries like Ramda, lodash/fp).
- Cleaner pipelines: `users.map(formatUser(currentLocale))`.

Q 3.13 Tail Recursion

A function call is in **tail position** if it's the last thing the function does — its return value is returned directly, with no further work. A function that only calls itself in tail position is **tail recursive**.

```
// NOT tail recursive – the multiplication happens AFTER the recursive call
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}

// Tail recursive – recursive call is the last operation
function factorialTR(n, acc = 1) {
  if (n <= 1) return acc;
  return factorialTR(n - 1, n * acc);
}
```

WHY IT MATTERS

In languages with **tail-call optimization (TCO)**, the engine can reuse the current stack frame for the recursive call instead of pushing a new one, avoiding stack overflow.

JS reality: the ES2015 spec mandated TCO in strict mode, but only Safari/JavaScriptCore implements it. V8 (Chrome/Node) and SpiderMonkey (Firefox) don't. Practically, convert deep recursion to iteration if you might blow the stack.

Q 3.14

Garbage Collection and Memory Management

JavaScript engines manage memory automatically. The garbage collector reclaims objects that are no longer **reachable** from the roots (globals, the call stack, etc.).

MODERN GC (V8)

- **Mark-and-sweep** — start from roots, mark reachable objects, sweep the unmarked ones.
- **Generational** — splits the heap into young (new, short-lived) and old (survivors). Young is collected often and quickly; old is collected less often.
- **Incremental / concurrent** — work is broken into small pieces or done off the main thread to avoid long pauses.

COMMON MEMORY LEAKS

- Lingering references in long-lived data structures (caches that never evict).
- Event listeners never removed from elements that *are* removed.
- Closures holding references to large objects longer than needed.
- Detached DOM nodes still referenced by JS.
- Timers (`setInterval`) that hold onto context.

Use `WeakMap` / `WeakSet` when you want to associate data with an object without preventing its collection.

Q 3.15

Generator Functions

A **generator** is a function that can be paused and resumed. Calling it returns an iterator; each `next()` runs until the next `yield`.

```
function* counter() {
  let i = 0;
  while (true) {
    yield i++;
  }
}

const c = counter();
c.next().value; // 0
c.next().value; // 1
c.next().value; // 2
```

USES

- Infinite or lazy sequences (Fibonacci, IDs).

- Building custom iterables (`[Symbol.iterator]`).
- State machines and coroutines.
- Async control flow (Redux-Saga uses generators to model side effects as effects).

```
async function* fetchPages(url) {
  let next = url;
  while (next) {
    const res = await fetch(next).then(r => r.json());
    yield res.items;
    next = res.next;
  }
}
for await (const items of fetchPages("/api/feed")) { /* ... */ }
```

Q 3.16 Promises

A **Promise** represents a value that will be available at some point — successfully or with an error. It has three states:

- **pending** — not yet settled.
- **fulfilled** — resolved with a value.
- **rejected** — failed with a reason.

Once settled, a promise's state and value are immutable.

```
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve(42), 100);
});

p.then(v => console.log(v))           // 42
  .catch(err => console.error(err))
  .finally(() => console.log("done"));

// async/await – same thing, easier to read
async function run() {
  try {
    const v = await p;
    console.log(v);
  } catch (err) {
    console.error(err);
  } finally {
    console.log("done");
  }
}
```


Always handle rejections. An `await` without `try/catch` bubbles errors up; a `.then()` chain without a final `.catch()` can become an unhandled rejection.

Q 3.17 OOP in JavaScript

JavaScript supports object-oriented programming through prototypes and the ES6 `class` syntax (which is sugar over prototypes).

THE FOUR PILLARS

- **Encapsulation** — bundle data + behavior. Private fields with `#`.
- **Inheritance** — `extends` chains prototypes together.
- **Polymorphism** — subclasses override parent methods.
- **Abstraction** — hide implementation details, expose a clean interface.

```
class Animal {
  #species; // private field
  constructor(name, species) {
    this.name = name;
    this.#species = species;
  }
  describe() { return `${this.name} (${this.#species})`; }
  static fromObject(o) { return new Animal(o.name, o.species); } // static
}

class Dog extends Animal {
  constructor(name) { super(name, "dog"); }
  describe() { return super.describe() + " - woof"; } // polymorphism
}

const d = new Dog("Rex");
d.describe(); // "Rex (dog) - woof"
d instanceof Animal; // true
```

Q 3.18 Shallow Copy vs Deep Copy

Aspect	Shallow copy	Deep copy
What's copied	Top-level properties only; nested objects share references.	Every level — fully independent clone.
Side-effect risk	Mutating nested data affects the original.	None — independent.

Aspect	Shallow copy	Deep copy
Cost	Cheap.	Expensive for large structures.

```
const orig = { a: 1, nested: { b: 2 } };

// Shallow copies
const s1 = { ...orig };
const s2 = Object.assign({}, orig);
s1.nested.b = 99;           // ALSO mutates orig.nested.b — same reference

// Deep copies
const d1 = structuredClone(orig);           // modern, built-in
const d2 = JSON.parse(JSON.stringify(orig)); // works only for JSON-safe data
const d3 = _.cloneDeep(orig);               // lodash
```

`structuredClone()` is the right answer in modern JS — handles dates, maps, sets, typed arrays, cyclic references. `JSON.parse(JSON.stringify(x))` drops `undefined`, functions, `Date`s, and breaks on cycles.

Q 3.19

Synchronous vs Asynchronous Functions

Aspect	Synchronous	Asynchronous
Execution	Runs to completion, blocking the call stack.	Starts the work and returns a Promise/callback; the rest runs later.
Return value	The actual value.	A Promise (or it uses a callback).
Blocking	Yes — UI freezes, event loop pauses.	No — other code runs while waiting.
Typical use	CPU work, pure computation.	Network, file I/O, timers, DB queries.

```
// Synchronous — blocks until done
const fs = require("fs");
const data = fs.readFileSync("./big.txt", "utf8");
console.log(data);

// Asynchronous (Promise)
const fsp = require("fs").promises;
const data2 = await fsp.readFile("./big.txt", "utf8");
console.log(data2);
```

```
// Asynchronous (callback – Node style)
fs.readFile("./big.txt", "utf8", (err, data) => { /* ... */ });
```

In a browser, sync I/O blocks the main thread and freezes the UI. In a Node server, sync I/O blocks ALL requests on that process — almost always wrong outside of one-shot scripts.

SECTION 04

TypeScript

12 questions

Q 4.01

What is `as const`?

`as const` is a **const assertion**. It tells TypeScript to infer the most specific (literal, readonly) type possible instead of widening to general types like `string`, `number`, or mutable arrays.

```
// Without as const – widened
const role = "admin";           // type: "admin" (because const binding)
const config = { mode: "dev" }; // type: { mode: string }
const arr = [1, 2, 3];          // type: number[]

// With as const – narrow + readonly
const config2 = { mode: "dev" } as const;
// type: { readonly mode: "dev" }

const arr2 = [1, 2, 3] as const;
// type: readonly [1, 2, 3]

const roles = ["admin", "user", "guest"] as const;
type Role = typeof roles[number]; // "admin" | "user" | "guest"
```

A favorite pattern: derive a union type from a runtime array of allowed values. One declaration, no drift between runtime and types.

Q 4.02

Private Access Modifiers

TypeScript offers two ways to mark class members as private — they look similar but are very different at runtime.

Feature	<code>private</code> (TS keyword)	<code>#</code> (ES private fields)
Enforcement	Compile-time only — erased at runtime.	True runtime privacy — JS engine enforces it.
Access from outside	Possible via <code>obj["field"]</code> or after compilation.	Throws <code>SyntaxError</code> .
Inherited classes	Cannot access (compile error).	Cannot access (runtime error).
Available in	TypeScript only.	Standard JS — ES2022+.

```
class WithTSPrivate {
  private secret = 42;
}
const a = new WithTSPrivate();
(a as any).secret; // 42 – TS private doesn't survive at runtime
```

```
class WithJSPrivate {
  #secret = 42;
}
const b = new WithJSPrivate();
// b.#secret           // SyntaxError outside the class
```

OTHER ACCESS MODIFIERS (TS-ONLY)

- `public` — default; visible everywhere.
- `protected` — visible to the class and its subclasses.
- `readonly` — can be set in the constructor, then immutable.

Q 4.03 Type Guards

A **type guard** is a runtime check that narrows a value's type within a block. TypeScript follows the narrowing through control flow.

BUILT-IN NARROWING

```
function format(value: string | number) {
  if (typeof value === "string") {
    return value.toUpperCase(); // value: string here
  }
  return value.toFixed(2);      // value: number here
}

function speak(animal: Dog | Fish) {
  if (animal instanceof Dog) {
    animal.bark();              // narrowed to Dog
  } else {
    animal.swim();              // narrowed to Fish
  }
}

if ("email" in user) { user.email; } // `in` operator narrows
```

USER-DEFINED TYPE GUARD

```
interface Cat { meow(): void; }
interface Dog { bark(): void; }

function isCat(a: Cat | Dog): a is Cat { // <-- return type predicate
  return (a as Cat).meow !== undefined;
}

function handle(a: Cat | Dog) {
  if (isCat(a)) {
    a.meow(); // narrowed to Cat
  }
}
```

```

    } else {
      a.bark();    // narrowed to Dog
    }
  }
}

```

DISCRIMINATED UNIONS

```

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number };

function area(s: Shape) {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;    // narrowed
    case "square": return s.side * s.side;
  }
}

```

Discriminated unions plus a literal "kind" field are the cleanest way to model "this OR that" data — exhaustive checks become a switch the compiler can verify.

Q 4.04 Structural vs Nominal Typing

Structural typing — types are compatible if their *shape* matches (same properties / signatures).

Nominal typing — types are compatible only if they have the same *name* / declaration.

TypeScript is **structurally typed**. C#, Java, Rust are **nominally typed**.

```

interface Point2D { x: number; y: number; }
class Coord { constructor(public x: number, public y: number) {} }

const fn = (p: Point2D) => p.x + p.y;
fn(new Coord(1, 2));    // OK — Coord has x and y, so it satisfies Point2D
fn({ x: 1, y: 2 });    // OK — plain object with matching shape

```

FAKING NOMINAL TYPING IN TS — BRANDED TYPES

```

type UserId = string & { readonly __brand: "UserId" };
type OrderId = string & { readonly __brand: "OrderId" };

function getUser(id: UserId) { /* ... */ }

const u = "u_123" as UserId;
const o = "o_456" as OrderId;
getUser(u);    // OK
getUser(o);    // Type error — different brand

```

Brands buy you nominal-style safety for primitive-looking IDs without changing the runtime representation.

Q 4.05 What is void and when to use it?

`void` is the return type of a function that doesn't return a useful value. It's distinct from `undefined`: a `void` function may still `return` early; a function returning `undefined` must explicitly return that value.

```
function log(msg: string): void {
  console.log(msg);
  // no return needed
}

function noReturn(): undefined {
  return undefined; // must explicitly return undefined
}
```

CALLBACKS AND THE VOID QUIRK

A `void`-returning function type accepts functions that return *anything*:

```
type Cb = (item: number) => void;

const cb: Cb = (n) => n * 2; // OK — return value is ignored

[1, 2, 3].forEach((n) => n.toString()); // forEach signature is void
```

Use `void` for callback return types when the caller will ignore the returned value. This avoids forcing implementers to add `return;` just to satisfy types.

Q 4.06 unknown vs never

	unknown	never
Meaning	"I don't know yet" — could be anything.	"This can't happen" — no value is ever of type <code>never</code> .
Assigning TO it	Anything is assignable.	Nothing is assignable.
Assigning FROM it		Assignable to anything.

	unknown	never
	Only to <code>unknown</code> or <code>any</code> — must narrow first.	
Typical use	Safer replacement for <code>any</code> at API boundaries (e.g. parsed JSON).	Functions that never return: throw, infinite loop. Exhaustiveness checks.

```
// unknown – safe any
function handle(x: unknown) {
  // x.toUpperCase()           // Error – must narrow
  if (typeof x === "string") {
    x.toUpperCase();           // OK
  }
}

// never – exhaustive switch check
type Shape = { kind: "circle" } | { kind: "square" };
function area(s: Shape) {
  switch (s.kind) {
    case "circle": return 1;
    case "square": return 2;
    default:
      const _exhaustive: never = s; // compile error if Shape gains a new kind
      throw new Error(_exhaustive);
  }
}
```

Q 4.07 Union Types (the `|` operator)

A **union** type means "this value can be one of several types". Read `A | B` as "A or B".

```
let value: string | number;
value = "hello"; // OK
value = 42;      // OK
value = true;    // Error
```

USE CASES

- Function arguments that accept multiple shapes: `function id(x: string | number)`.
- Modeling state machines: `type Status = "idle" | "loading" | "success" | "error"`.
- Discriminated unions for clean exhaustive handling.

```
type Result<T> =
  | { ok: true; value: T }
  | { ok: false; error: string };
```

```
function handle<T>(r: Result<T>) {
  if (r.ok) {
    return r.value;          // narrowed – value exists here
  }
  console.error(r.error); // narrowed – error exists here
}
```

Union of literal strings is TypeScript's lightweight enum. Combine with `as const` arrays to derive both runtime list and type.

Q 4.08 Mixins in TypeScript

Mixins are a pattern for composing behavior onto classes by combining multiple sources, instead of using single-class inheritance. TypeScript supports them via class factories that take a base class and return a subclass.

```
type Constructor<T = {}> = new (...args: any[]) => T;

// Mixin factory: adds an eat() method to any class
function CanEat<TBase extends Constructor>(Base: TBase) {
  return class extends Base {
    eat() { console.log("Eating..."); }
  };
}

function CanWalk<TBase extends Constructor>(Base: TBase) {
  return class extends Base {
    walk() { console.log("Walking..."); }
  };
}

class Animal {
  constructor(public name: string) {}
}

const Person = CanEat(CanWalk(Animal));

const p = new Person("Alice");
p.eat(); // Eating...
p.walk(); // Walking...
console.log(p.name); // Alice
```

WHY USE MIXINS

- TypeScript only supports single-class inheritance — mixins simulate multiple inheritance.
- Compose reusable behaviors (loggable, serializable, taggable) across unrelated class hierarchies.
- Favored by frameworks like Vue (mixins option, now mostly replaced by composition).

Modern designs often prefer composition (small classes injected via constructor) or hooks (in React) over mixins — they're more explicit about behavior provenance.

Q 4.09 Type Assertions

A **type assertion** tells the compiler "trust me, I know what this is" — it overrides the inferred type without any runtime check.

```
// 1. as syntax (preferred, works in .tsx)
const el = document.getElementById("app") as HTMLDivElement;

// 2. Angle-bracket syntax (doesn't work in JSX/TSX)
const el2 = <HTMLDivElement>document.getElementById("app");

// Non-null assertion – "I promise this isn't null/undefined"
const el3 = document.getElementById("app")!;
el3.innerHTML = "...";

// Double assertion – escape hatch when types are very different
const num = ("123" as unknown) as number;
```

ASSERTION VS CASTING

An assertion is a *compile-time only* declaration — it does NOT convert the runtime value. If you assert a string is a number, you'll crash at runtime.

```
const x = "5" as unknown as number;
x.toFixed(2); // Compiles, but throws at runtime: x is actually a string
```

Use assertions sparingly. Most cases that look like they need an assertion can be solved with a type guard or a better type signature instead.

Q 4.10 Template Literal Types

Template literal types let you build new string-literal types by interpolating other types — just like JS template literals at runtime, but at the type level.

```
type Greeting = `Hello, ${string}`;
let g: Greeting;
g = "Hello, Alice"; // OK
g = "Hi, Bob";      // Error – must start with "Hello, "
```

COMBINING WITH UNIONS

```
type Size = "small" | "medium" | "large";
type Color = "red" | "blue";

type Shirt = `${Size}-${Color}`;
// "small-red" | "small-blue" | "medium-red" | "medium-blue" | "large-red" | "large-blue"

let s: Shirt;
s = "small-red";           // OK
s = "small-green";         // Error
```

REAL-WORLD USES

- Strict event names (`on${Capitalize<EventName>}``).
- CSS-in-JS prop names (`--${string}`` custom properties).
- Route patterns (`/users/${string}``).
- Branded IDs derived from literals.

TypeScript ships built-in intrinsic types: `Uppercase<S>` , `Lowercase<S>` , `Capitalize<S>` , `Uncapitalize<S>` for transforming string literal types.

Q 4.11 Decorators

Decorators are functions that can be attached to a class, method, property, or accessor to add metadata or modify behavior at *design time*. Heavily used by frameworks like Angular, NestJS, and TypeORM.

Two flavors exist: legacy *experimental decorators* (enabled with `"experimentalDecorators": true` in `tsconfig.json`) and the standardized *ES decorators* (TS 5.0+). Examples below use the legacy form, which is what most frameworks still use.

CLASS DECORATOR

```
function sealed(constructor: Function) {
  Object.seal(constructor);
  Object.seal(constructor.prototype);
}

@sealed
class BugReport {
  type = "report";
  title: string;
```

```
constructor(t: string) { this.title = t; }
}
```

METHOD DECORATOR

```
function log(target: any, key: string, desc: PropertyDescriptor) {
  const original = desc.value;
  desc.value = function (...args: any[]) {
    console.log(`call ${key}(${args})`);
    return original.apply(this, args);
  };
}

class Calculator {
  @log
  add(a: number, b: number) { return a + b; }
}
```

PROPERTY & PARAMETER DECORATORS

- **Property decorator** — add metadata (e.g. validation rules, ORM column info).
- **Parameter decorator** — usually paired with `reflect-metadata` for DI (NestJS `@Inject`).

For a NestJS-style DI app, decorators are how the framework knows what to inject and where to register routes. The `emitDecoratorMetadata` compiler option emits constructor parameter types so reflection libraries can resolve them.

Q 4.12 Parameter Properties (constructor shorthand)

TypeScript lets you declare and initialize a class field directly in the constructor's parameter list by prefixing it with an access modifier (`public` , `private` , `protected`) or `readonly` . This is called a **parameter property**.

```
// Without parameter properties – verbose
class Person {
  public name: string;
  private age: number;
  constructor(name: string, age: number) {
    this.name = name;
    this.age = age;
  }
}

// With parameter properties – same thing, one line
class Person2 {
  constructor(
    public name: string,
```

```
    private age: number,  
    readonly id: string,  
  ) {}  
}  
  
const p = new Person2("Dincho", 28, "u-1");  
p.name;    // OK – public  
p.id;      // OK – readonly, can read  
// p.age   // Error – private  
// p.id = "u-2" // Error – readonly
```

WHY IT'S USEFUL

- Eliminates the repetitive "declare field / assign in constructor" pattern.
- Reads cleanly with dependency injection (NestJS, Angular) where every dependency is constructor-injected.

`readonly` can be combined with the others (`public readonly` , `private readonly`). The field becomes assignable in the constructor only; reads work everywhere the access level permits.

SECTION 05

React

25 questions

Q 5.01

What is JSX?

JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside JS code. It's transpiled (by Babel or the TS compiler) into `React.createElement()` calls — or, in modern React, into the JSX runtime's `jsx()` call.

```
const el = <h1>Hello, {name}!</h1>;

// Compiles roughly to:
const el = React.createElement("h1", null, "Hello, ", name, "!");
```

KEY RULES

- Embed JS expressions with `{ ... }`.
- A component must return a single root element — use Fragments (`<>...</>`) when you don't want extra DOM.
- HTML attributes become camelCase: `className`, `onClick`, `tabIndex`.
- Self-closing tags require `/>`: `<img />`.

Q 5.02

Functional vs Class Components

Feature	Function component	Class component
Syntax	Plain function returning JSX	Class extending <code>React.Component</code>
State	<code>useState</code> , <code>useReducer</code>	<code>this.state</code> + <code>this.setState</code>
Lifecycle	<code>useEffect</code> , <code>useLayoutEffect</code>	<code>componentDidMount</code> etc.
<code>this</code>	Not needed	Required; needs binding
Usage today	Preferred in all new code	Legacy, still supported

```
function Greeting({ name }: { name: string }) {
  return <h1>Hello, {name}!</h1>;
}

class GreetingClass extends React.Component<{ name: string }> {
  render() { return <h1>Hello, {this.props.name}!</h1>; }
}
```


Q 5.03

Props vs State

Feature	Props	State
Source	Passed in from the parent	Managed inside the component
Mutability	Read-only (from the child's view)	Mutable via setter
Triggers re-render	Parent passing new value	Setter called with new value
Typical use	Configuration, callbacks, data flowing down	Internal data that changes over time

```
function Counter({ title }: { title: string }) {           // prop
  const [count, setCount] = React.useState(0);             // state
  return (
    <div>
      <p>{title}</p>
      <button onClick={() => setCount(c => c + 1)}>{count}</button>
    </div>
  );
}
```

Props are the inputs to a component. State is its internal memory.

Q 5.04

Controlled vs Uncontrolled Components

Aspect	Controlled	Uncontrolled
Source of truth	React state	The DOM
Read value via	<code>value</code> prop + <code>onChange</code>	<code>ref</code>
Use when	Live validation, computed values, syncing with other state	Simple forms, file inputs, integrating non-React widgets

```
// Controlled
function Form() {
  const [name, setName] = React.useState("");
  return <input value={name} onChange={e => setName(e.target.value)} />;
}

// Uncontrolled
```

```
function Form2() {
  const inputRef = React.useRef<HTMLInputElement>(null);
  const submit = () => console.log(inputRef.current?.value);
  return <input ref={inputRef} />;
}
```

File inputs (`<input type="file">`) are always uncontrolled — their value can't be set programmatically for security reasons.

Q 5.05 The key attribute in lists

`key` gives React a stable identity for each item in a list so it can match elements across renders and update the DOM efficiently.

```
{users.map(u => (
  <li key={u.id}>{u.name}</li>
))}
```

RULES

- Keys must be unique *among siblings* — not globally.
- Use stable IDs from your data, not the array index, when items can be inserted, removed, or reordered.
- Don't generate keys at render time (`Math.random()`) — that defeats the whole point.

Without proper keys, React falls back to position-matching, which causes bugs like form inputs swapping values when a list is reordered.

Q 5.06 Virtual DOM

The **Virtual DOM** is React's in-memory tree of JavaScript objects describing what the UI should look like. Direct DOM mutations are expensive; comparing two JS objects is cheap.

HOW A RENDER WORKS

1. React calls your components and builds a new Virtual DOM tree.
2. It **diffs** the new tree against the previous one.
3. It computes the minimal set of real DOM mutations needed.
4. It commits those mutations to the real DOM in one pass.

WHY IT'S FAST

- Batched DOM updates instead of one-at-a-time mutations.
- Heuristic diffing in $O(n)$ using keys and component types.
- Lets you write declaratively (describe the UI) instead of imperatively (manipulate the DOM).

The Virtual DOM concept and React's Fiber implementation are different things — see the Fiber question for how scheduling layers on top.

Q 5.07

Lifecycle Methods (Class Components)

Phase	When it runs	Methods
Mounting	Component is being inserted into the DOM	<code>constructor</code> → <code>getDerivedStateFromProps</code> → <code>render</code> → <code>componentDidMount</code>
Updating	Props or state changed	<code>getDerivedStateFromProps</code> → <code>shouldComponentUpdate</code> → <code>render</code> → <code>getSnapshotBeforeUpdate</code> → <code>componentDidUpdate</code>
Unmounting	Being removed from the DOM	<code>componentWillUnmount</code>
Error	Child threw during rendering	<code>getDerivedStateFromError</code> , <code>componentDidCatch</code>

In function components, all of this is expressed with hooks — primarily `useEffect` (cleanup function = unmount), `useLayoutEffect` (sync after DOM mutation), and Error Boundaries (still class-only).

Q 5.08

React Hooks — overview

Hooks let function components use React features (state, lifecycle, context, refs) that previously required classes. Introduced in React 16.8.

Hook	What it does
<code>useState</code>	Local state.
<code>useEffect</code>	Side effects after render (data fetching, subscriptions).
<code>useLayoutEffect</code>	Same as <code>useEffect</code> but synchronous, before paint.
<code>useContext</code>	Subscribe to a React context.

Hook	What it does
<code>useReducer</code>	State with a reducer (useful for complex transitions).
<code>useRef</code>	Mutable value that persists across renders; also DOM refs.
<code>useMemo</code>	Cache an expensive computed value.
<code>useCallback</code>	Cache a function reference between renders.
<code>useTransition</code> / <code>useDeferredValue</code>	Mark updates as non-urgent (concurrent features).
<code>useId</code>	Stable unique IDs for SSR-safe a11y attributes.

RULES OF HOOKS

- Only call hooks *at the top level* — never inside loops, conditions, or nested functions.
- Only call hooks from React function components or custom hooks (whose name starts with `use`).

Q 5.09 How Hooks Work Under the Hood

Hooks rely on the fact that React calls each function component in a known, deterministic order. React maintains a per-component, ordered linked list of hook "slots", and walks it on every render.

MENTAL MODEL

```
// Roughly:
let currentComponent = { hooks: [], index: 0 };

function useState(initial) {
  const i = currentComponent.index++;
  const hooks = currentComponent.hooks;
  hooks[i] ??= { value: initial };
  const setValue = (v) => {
    hooks[i].value = typeof v === "function" ? v(hooks[i].value) : v;
    scheduleRender(currentComponent);
  };
  return [hooks[i].value, setValue];
}
```

This is why the **rules of hooks** exist: if you call a hook conditionally, the order shifts between renders and React reads the wrong slot.

STORAGE IN FIBER

In the real implementation, each component instance has a Fiber node; each Fiber holds a linked list of **Hook** objects (**memoizedState** , **queue** , **next**). The order in which the function calls hooks determines which node it reads.

Q 5.10 Context API

Context lets you pass data through the component tree without prop-drilling every level. Useful for theme, current user, locale, feature flags — values that many components need.

```
const ThemeContext = React.createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() { return <ThemedButton />; }

function ThemedButton() {
  const theme = React.useContext(ThemeContext);
  return <button className={theme}>OK</button>;
}
```

Performance note: every consumer re-renders when the context value changes. For frequently changing values, prefer a state library (Redux, Zustand, Jotai) or split context into smaller pieces.

Q 5.11 Higher-Order Components (HOCs)

A **HOC** is a function that takes a component and returns a new component, usually with additional props or behavior. It's a pattern for cross-cutting concerns — older than hooks and largely superseded by them.

```
function withLogger<P>(Wrapped: React.ComponentType<P>) {
  return function WithLogger(props: P) {
    React.useEffect(() => { console.log("mounted"); }, []);
    return <Wrapped {...props} />;
  };
}
```

```
const LoggedButton = withLogger(Button);
```

MODERN EQUIVALENT

Most HOCs are now better expressed as **custom hooks** (state + behavior) or **render props**. Hooks compose more cleanly and don't add wrapper components to the tree.

HOCs you'll still see in the wild: `connect()` from react-redux, `withRouter` from older React Router, `withTranslation` from i18next. New code generally uses the hook equivalents (`useSelector`, `useParams`, `useTranslation`).

Q 5.12 Reconciliation

Reconciliation is the algorithm React uses to figure out what to change in the DOM when a component re-renders. It compares the previous Virtual DOM tree to the new one and produces the minimal mutations.

THE HEURISTICS

- **Same type, same position** — reuse the DOM node, update props.
- **Different type** — destroy the old subtree, build the new one from scratch.
- **List items with keys** — match by key across renders to detect inserts, removes, and reorders.

WHY IT'S O(N) INSTEAD OF O(N³)

A general tree-diff is $O(n^3)$. React makes two assumptions that bring it down to $O(n)$:

1. Two elements of different types produce different trees (no cross-type matching).
2. Keys, when provided, are stable identities — so list diffing is local instead of full pairwise.

Q 5.13 React Portals

A **portal** renders children into a DOM node outside the parent's DOM hierarchy, while keeping them in the React tree for events, context, and state.

```
function Modal({ children }: { children: React.ReactNode }) {
  return ReactDOM.createPortal(
    <div className="modal">{children}</div>,
    document.body, // render into <body>, not the parent
  );
}
```

WHEN TO USE

- Modals, tooltips, popovers — escape ancestor `overflow: hidden` or stacking contexts.
- Toast notifications anchored to a global root.
- Floating UI that needs to position relative to the viewport, not its parent.

Events still bubble through the React tree, not the DOM tree — so a click inside the portal is still handled by handlers on the parent component.

Q 5.14 React Strict Mode

`<React.StrictMode>` is a development-only wrapper that helps catch problems early. It does nothing in production.

WHAT IT DOES IN DEV

- **Double-invokes** render bodies, effects, and reducers to expose impure logic.
- Warns about deprecated APIs (legacy refs, `findDOMNode`, etc.).
- Warns when components have side effects in unsafe lifecycle methods.
- Helps spot effects that aren't safe to run twice (missing cleanup, server-side state that doesn't get reset).

```
<React.StrictMode>
  <App />
</React.StrictMode>
```

Double-invocation can be confusing the first time — if your effect runs twice in dev but once in prod, that's expected. Treat it as a feature: it surfaces effects that aren't idempotent.

Q 5.15 The Spread Operator in React

The spread operator (`...`) appears constantly in React for forwarding props, immutable updates, and merging objects.

SPREADING PROPS

```
function Button({ children, ...rest }: ButtonProps) {
  return <button {...rest}>{children}</button>;
}
```

IMMUTABLE STATE UPDATES

```
// Object – replace one field
setUser(prev => ({ ...prev, name: "New" }));

// Array – append, prepend, remove without mutating
setItems(prev => [...prev, newItem]);
setItems(prev => prev.filter(x => x.id !== removedId));
```

Watch out: spread is a *shallow* copy. Nested objects are still shared by reference — mutating `copy.profile.age` will mutate the original too. Use `structuredClone` or a library like Immer when you need deep immutability.

Q 5.16 Conditional Rendering

React has no special syntax for conditionals — you use regular JS expressions inside JSX. The common patterns:

```
// 1. if / early return
if (loading) return <Spinner />;
return <Content />;

// 2. Ternary
{isOpen ? <Modal /> : null}

// 3. Logical &&
{hasError && <ErrorBanner />}

// 4. Lookup / map
const variants = { success: SuccessIcon, error: ErrorIcon, info: InfoIcon };
const Icon = variants[status];
return <Icon />;

// 5. IIFE for complex logic
{(() => {
  switch (status) {
    case "loading": return <Spinner />;
    case "error":   return <ErrorView />;
    default:       return <Content />;
  }
})()}}
```

Gotcha with `&&`: if the left side is a falsy number like `0`, React will render `0` to the DOM. Use `Boolean(x) && ...` or a ternary when the value might be a number.

Q 5.17 **useEffect Dependency Array**

The second argument to `useEffect` controls when the effect runs.

Dep array	When the effect runs
Omitted	After every render.
<code>[]</code>	Only once, after the first render.
<code>[a, b]</code>	After the first render, and whenever <code>a</code> or <code>b</code> changes (by <code>Object.is</code>).

```
React.useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup runs before next effect & on unmount
}, [tick]);
```

COMMON PITFALLS

- **Missing deps** — referencing state or props without listing them leads to stale closures. The `react-hooks/exhaustive-deps` ESLint rule catches this.
- **Object/array deps** — a new `{...}` each render means the effect runs every time. Memoize with `useMemo`, or split into primitive deps.
- **Function deps** — same problem; either move the function inside the effect or wrap it in `useCallback`.

Q 5.18 **useReducer**

`useReducer` is an alternative to `useState` for state that has multiple related fields or complex transitions. It models state as `(state, action) => newState` — the Redux pattern, scoped to one component.

```
type State = { count: number; step: number };
type Action =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "setStep"; value: number };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment": return { ...state, count: state.count + state.step };
    case "decrement": return { ...state, count: state.count - state.step };
    case "setStep": return { ...state, step: action.value };
  }
}
```

```
function Counter() {
  const [state, dispatch] = React.useReducer(reducer, { count: 0, step: 1 });
  return (
    <>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: "increment" })}>+</button>
    </>
  );
}
```

Pick `useReducer` over `useState` when you have several pieces of related state that transition together, or when next state depends on the previous in non-trivial ways.

Q 5.19 React.lazy and Suspense

`React.lazy` and `Suspense` together implement **code splitting** — load a component's bundle only when it's actually used.

```
const Heavy = React.lazy(() => import("./Heavy"));

function App() {
  return (
    <React.Suspense fallback={<Spinner />}>
      <Heavy />
    </React.Suspense>
  );
}
```

- `React.lazy(loader)` takes a function that returns a dynamic `import()`. The bundler emits a separate chunk for that module.
- `Suspense` catches the loading state from any lazy descendant and renders the fallback until it resolves.
- Use one `Suspense` boundary per "feature" you want to load independently — too coarse and the whole page shows a spinner; too fine and the UI jitters.

React 18 extended `Suspense` to support data fetching (used by frameworks like Next.js App Router and Relay), not just code-splitting.

Q 5.20

Optimizing Performance in React Apps

| MEASURE FIRST

React DevTools Profiler shows commit times and which components rendered. Don't optimize blindly — find the actual hot spot.

| COMMON LEVERS

- **Avoid unnecessary work in render** — pure computations only; move side effects to `useEffect`.
- **Memoize expensive values** with `useMemo` — only worth it when the computation is actually heavy or the result is used as a stable reference downstream.
- **Stable callbacks** with `useCallback` when passing to memoized children or as deps of effects.
- **React.memo** on components that re-render with the same props (after measuring — it adds prop comparison overhead).
- **List virtualization** for long lists (`react-window` , `react-virtuoso`).
- **Code split** with `React.lazy` + `Suspense` at route or feature boundaries.
- **Move state down** — keep state as local as possible to limit the re-render radius.
- **Split context** or use a state library when many consumers re-render on every change.
- **Concurrent features** — `useTransition` / `useDeferredValue` to mark heavy updates as non-urgent.

The trap: sprinkling `useMemo` and `React.memo` everywhere often makes things slower, not faster. Memoization isn't free — measure both before and after.

Q 5.21

Reactive Programming in React

Reactive programming means modeling your app as data streams and reactions: UI is a function of state, and state changes propagate automatically to anything that depends on it.

| HOW REACT EMBODIES IT

- The render function takes props and state and returns UI — declarative `view = f(state)`.
- State changes (via setters or dispatch) propagate through the component tree.
- `useEffect` reacts to dependency changes.
- Derived state is computed in render or with `useMemo`, not stored separately.

| ADDING OBSERVABLE STREAMS

For pull-based "react to a stream over time", libraries integrate with React:

- **RxJS** — observables; manually subscribe in `useEffect`, or use `react-rxjs`.

- **MobX** — reactive observables that wrap components in `observer()` to auto-track reads.
- **Signals** (Preact Signals, TC39 proposal) — finer-grained reactivity than React's render-the-tree approach.
- **TanStack Query / SWR** — reactive async state for server data.

Q 5.22 Passing Data from Child to Parent

React's data flow is one-way (top-down). To send data *up*, the parent passes a callback down and the child invokes it.

```
function Parent() {
  const [value, setValue] = React.useState("");
  return <Child onChange={setValue} />;
}

function Child({ onChange }: { onChange: (v: string) => void }) {
  return <input onChange={e => onChange(e.target.value)} />;
}
```

OTHER OPTIONS

- **Lifted state** — keep shared state in the closest common ancestor and pass setters down.
- **Context** — for state needed by many descendants; expose both value and setter.
- **State libraries** (Redux, Zustand, Jotai) — for app-wide state any component can read or write.
- **Refs + `useImperativeHandle`** — for imperative APIs (focus, scroll), not regular data flow.

Q 5.23 Why can't `useEffect` be async?

The function you pass to `useEffect` is expected to return either nothing or a **cleanup function**. An `async` function always returns a `Promise` — and React doesn't know what to do with a promise as a cleanup.

```
// WRONG
React.useEffect(async () => {
  const data = await fetch("/api"); // returns a Promise
  setData(data);
}, []);

// RIGHT — wrap an inner async function
React.useEffect(() => {
  let cancelled = false;
  (async () => {
    const res = await fetch("/api");
    const data = await res.json();
  })();
}, []);
```

```

    if (!cancelled) setData(data);
  })();
  return () => { cancelled = true; };    // cleanup
}, []);

```

Track an `AbortController` in the cleanup if you want to actually cancel in-flight requests, not just ignore late responses.

Q 5.24 React Fiber vs the Virtual DOM

The Virtual DOM is the *concept* — an in-memory representation of the UI that React diffs to compute DOM mutations. **Fiber** is the modern *implementation* of that idea, introduced in React 16.

WHAT FIBER ADDS

- **Incremental rendering** — work is split into units that can pause, resume, or abort. Long renders no longer block the main thread for tens of milliseconds.
- **Prioritization** — urgent updates (user input) jump ahead of background updates (data fetches, transitions).
- **Concurrent rendering** — multiple versions of the tree can be prepared in parallel; only the most up-to-date is committed.
- **Better error handling** — Error Boundaries.

FIBER DATA STRUCTURE

Each Fiber node is a JS object representing one unit of work: a component instance, a DOM element, etc. Fibers link to `child`, `sibling`, and `return` (parent), forming a tree you can traverse iteratively instead of recursively — that's what makes pausing possible.

From an interview answer: "VDOM is what React diffs; Fiber is the scheduler and reconciler that does the diffing in interruptible chunks, with priorities."

Q 5.25 How React Decides When to Re-render

A component re-renders when one of these happens:

- Its **state** changes (`useState` setter, `useReducer` dispatch).
- Its **props** change (parent re-rendered and passed new props).
- A **context** it consumes changes value.
- Its **parent re-rendered** — by default, React re-renders the whole subtree.

BAIL-OUTS

- Calling a setter with the **same value** (by `Object.is`) — React bails out and doesn't re-render.
- `React.memo(Comp)` — skips re-render if previous props equal new props (shallow comparison by default; supply a custom comparator if needed).
- `useMemo` / `useCallback` — keep stable references between renders so memoized children's prop check passes.

COMMON MISCONCEPTION

"Children don't re-render if their props haven't changed." Not true by default. Without `React.memo`, a child re-renders whenever its parent does, even if props are byte-identical. Memo is the opt-in optimization.

SECTION 06

Redux

9 questions

Q 6.01

What is Redux and How is it Structured?

Redux is a predictable state container. It centralizes app state in a single **store**, and state is read-only — the only way to change it is by dispatching **actions** that are handled by pure **reducer** functions.

THREE PRINCIPLES

- **Single source of truth** — one store for the whole app.
- **State is read-only** — describe changes as plain action objects.
- **Changes via pure functions** — reducers take (state, action) and return new state, without mutation.

CORE PIECES

- **Store** — holds state; exposes `getState()`, `dispatch()`, `subscribe()`.
- **Actions** — plain objects: `{ type: "todos/added", payload: {...} }`.
- **Reducers** — `(state, action) => newState`; combined into a root reducer.
- **Selectors** — functions that read derived data from state.
- **Middleware** — intercept dispatched actions for logging, async, etc.

Modern Redux uses Redux Toolkit (RTK). `createSlice` generates actions + reducer; `configureStore` sets up the store with sensible defaults; RTK Query handles server cache.

Q 6.02

Data Flow in React + Redux

1. A component reads state via `useSelector` (or `connect`'s `mapStateToProps`).
2. User interaction calls `dispatch(action)`.
3. The action flows through any middleware (thunks, sagas, logger).
4. The root reducer receives the action and computes new state.
5. The store notifies subscribers; `useSelector` compares the selected slice and re-renders if it changed.

```
const count = useSelector((s: RootState) => s.counter.value);
const dispatch = useDispatch();

<button onClick={() => dispatch(increment())}>{count}</button>
```

Selectors should return stable references for objects/arrays — use `createSelector` from `reselect` to memoize derived data and avoid unnecessary re-renders.

Q 6.03

The Role of the Store

The **store** is the runtime container for Redux state. There's one per app.

- Holds the entire state tree.
- `getState()` — returns the current state.
- `dispatch(action)` — sends an action through middleware and reducers.
- `subscribe(listener)` — registers a callback for state changes (react-redux uses this internally).
- `replaceReducer()` — used for code splitting and HMR.

```
import { configureStore } from "@reduxjs/toolkit";

const store = configureStore({
  reducer: { todos: todosReducer, user: userReducer },
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Q 6.04

The Role of a Reducer

A **reducer** is a pure function `(state, action) => newState` that describes how state changes in response to an action.

RULES

- Must be pure — no side effects, no mutation, no random data.
- Same inputs always produce the same output.
- Return a new state object (or the same one, unchanged).

```
// Old-school
function counter(state = 0, action) {
  switch (action.type) {
    case "increment": return state + 1;
    case "decrement": return state - 1;
    default: return state;
  }
}

// RTK — looks like mutation but uses Immer under the hood
const slice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment(s) { s.value += 1; }, // Immer makes this safe
  }
});
```

```
    incrementBy(s, action: PayloadAction<number>) { s.value += action.payload; },
  },
});
```

`combineReducers` stitches per-slice reducers into a root reducer; each slice owns one top-level key of state.

Q 6.05

Local Component State vs Redux State

Use local (useState/useReducer) when	Use Redux when
Only one component cares	Many components across the tree need it
Ephemeral UI (open/closed, hover)	App-wide concerns (current user, cart, notifications)
Easy to lift to a parent if needed	Cross-cutting state that survives navigation
You want minimal ceremony	You need time-travel debugging, middleware, or a clear action log

Heuristic: start with `useState`. Lift to a parent when a sibling needs it. Reach for Context for tree-wide read-only data (theme, user). Use Redux/Zustand/Jotai only when you genuinely have global state with multiple writers — otherwise it's overkill.

Q 6.06

Provider — How Components Get State

react-redux's `<Provider>` wraps your app and exposes the store via React context. Hooks (`useSelector`, `useDispatch`) read from that context.

```
import { Provider } from "react-redux";
import { store } from "./store";

ReactDOM.createRoot(document.getElementById("root")!).render(
  <Provider store={store}>
    <App />
  </Provider>,
);

// Inside any descendant:
const value = useSelector((s: RootState) => s.counter.value);
const dispatch = useDispatch<AppDispatch>();
```

react-redux subscribes to the store once per `useSelector` hook and uses a custom-shallow comparison to decide when to re-render — much more efficient than a plain context consumer for many subscribers.

Q 6.07 `connect()` — the Pre-Hooks API

`connect()` is a HOC from react-redux that wires a component to the store via `mapStateToProps` and `mapDispatchToProps`. The pre-hooks pattern.

```
function Counter({ value, increment }) {
  return <button onClick={increment}>{value}</button>;
}

const mapStateToProps = (state) => ({ value: state.counter.value });
const mapDispatchToProps = { increment };

export default connect(mapStateToProps, mapDispatchToProps)(Counter);
```

Modern code uses hooks (`useSelector`, `useDispatch`) — they compose better and don't add HOC wrappers. `connect()` still works and is common in older codebases.

Q 6.08 Redux Middleware

Middleware sits between `dispatch` and the reducer. It can inspect actions, dispatch new ones, delay them, or run side effects.

```
// A minimal middleware
const logger = (store) => (next) => (action) => {
  console.log("dispatching", action);
  const result = next(action);
  console.log("next state", store.getState());
  return result;
};

const store = configureStore({
  reducer,
  middleware: (getDefault) => getDefault().concat(logger),
});
```

COMMON MIDDLEWARE

- **redux-thunk** — lets action creators return functions for async work (built into RTK).
- **redux-saga** — async via generator-based "sagas".

- **redux-observable** — async via RxJS streams.
- **redux-logger** — pretty-prints actions and state to the console.
- **RTK Query** — generated middleware for caching server data.

Q 6.09**Redux Thunk**

A **thunk** is a function that wraps an async operation and dispatches actions describing its progress.

`redux-thunk` middleware allows action creators to return a function instead of a plain action.

```
// Manual thunk
function fetchUser(id: string) {
  return async (dispatch, getState) => {
    dispatch({ type: "user/loading" });
    try {
      const res = await fetch(`/api/users/${id}`);
      const user = await res.json();
      dispatch({ type: "user/loaded", payload: user });
    } catch (err) {
      dispatch({ type: "user/error", payload: err.message });
    }
  };
}

// RTK — createAsyncThunk generates pending/fulfilled/rejected actions for you
export const fetchUser = createAsyncThunk(
  "user/fetch",
  async (id: string) => {
    const res = await fetch(`/api/users/${id}`);
    return res.json();
  },
);
```

Thunks are great for simple async (load on mount, save on submit). For complex flows — cancellation, debouncing, race conditions — sagas or RTK Query are better suited.

SECTION 07

Next.js

14 questions

Q 7.01

What is Next.js?

Next.js is a React framework for production. It adds the things React leaves out: routing, server rendering, data fetching conventions, image and font optimization, API routes, and a build/deploy pipeline.

| KEY FEATURES

- **Rendering strategies** — SSR, SSG, ISR, CSR, streaming, plus React Server Components (App Router).
- **File-based routing** — folder structure defines routes; `app/` (modern) or `pages/` (legacy).
- **API routes** — server endpoints colocated with the frontend.
- **Image optimization** — `next/image` resizes, formats (AVIF/WebP), and lazy-loads.
- **Built-in CSS / Tailwind / Sass support.**
- **Middleware** at the edge for auth, redirects, A/B tests.

Q 7.02

SSR vs SSG

Feature	SSR (Server-Side Rendering)	SSG (Static Site Generation)
When HTML is built	Per request, on the server	At build time, once
Freshness	Always current	Stale until next build (unless ISR)
Performance	Slower TTFB; needs a server	CDN-fast; just files
Scaling	CPU per request	Trivial — static hosting
Use for	Personalized pages, dashboards, frequently changing data	Marketing pages, blogs, docs, product catalogs

Pages Router APIs: `getServerSideProps` for SSR, `getStaticProps` for SSG. App Router does this via Server Components and the fetch caching options (`{ cache: "no-store" }` vs default static).

Q 7.03

File-Based Routing

Next.js maps the file system to routes. No router configuration needed.

| APP ROUTER (MODERN)

```
app/
  page.tsx      -> /
  about/page.tsx -> /about
```

```

blog/[slug]/page.tsx  -> /blog/:slug
shop/
  layout.tsx          -> shared layout for /shop/*
  page.tsx             -> /shop
  [id]/page.tsx       -> /shop/:id
api/route.ts          -> /api (handler)

```

SPECIAL FILES

- `layout.tsx` — wraps a route subtree; preserved across navigation.
- `page.tsx` — the route's UI.
- `loading.tsx` — Suspense fallback while data resolves.
- `error.tsx` — error boundary.
- `not-found.tsx` — 404.
- `route.ts` — server-only API handler.

Pages Router (`pages/`) uses the same idea but with a single file per route and no nested layouts.

Q 7.04 `getStaticProps` (Pages Router / SSG)

Runs at build time on the server. Returns props that get baked into the static HTML.

```

// pages/blog/[slug].tsx
export async function getStaticProps({ params }) {
  const post = await fetchPost(params.slug);
  return {
    props: { post },
    revalidate: 60, // ISR – regenerate at most once a minute
  };
}

export async function getStaticPaths() {
  const slugs = await fetchAllSlugs();
  return {
    paths: slugs.map(slug => ({ params: { slug } })),
    fallback: "blocking", // unknown slugs: SSR at first hit, then cached
  };
}

```

App Router equivalent: a Server Component with `fetch()` that's statically cached by default. Add `export const revalidate = 60;` for ISR semantics.

Q 7.05

getServerSideProps (Pages Router / SSR)

Runs on every request on the server. Returns props for the React page rendered per request.

```
// pages/dashboard.tsx
export async function getServerSideProps({ req, query }) {
  const session = await getSession(req);
  if (!session) {
    return { redirect: { destination: "/login", permanent: false } };
  }
  const data = await fetchUserData(session.userId);
  return { props: { data } };
}
```

Each request runs Node code, so the page can't be served from a CDN cache. Use when content is per-user or changes per request. App Router: a Server Component with `fetch(url, { cache: "no-store" })`.

Q 7.06

getStaticPaths

Paired with `getStaticProps` for dynamic SSG routes. Tells Next.js which paths to pre-render at build time.

```
export async function getStaticPaths() {
  const products = await fetchProducts();
  return {
    paths: products.map(p => ({ params: { id: p.id.toString() } })),
    fallback: "blocking", // or true (show fallback UI), or false (404 unknown)
  };
}
```

fallback	Behavior for unknown paths
false	Return 404 — only pre-built paths exist.
true	Serve a fallback UI immediately; render in background; cache for future requests.
"blocking"	SSR the page on first hit, then cache as static — no fallback UI.

Q 7.07

Dynamic Routes

Use brackets in the file/folder name to capture a URL segment as a parameter.


```

app/blog/[slug]/page.tsx          -> /blog/hello-world
app/products/[category]/[id]/page.tsx -> /products/shoes/123
app/docs/[...slug]/page.tsx       -> /docs/a/b/c    (catch-all)
app/docs/[...slug]/page.tsx       -> /docs and /docs/a/b  (optional catch-all)

```

```

// App Router
export default function Page({ params }: { params: { slug: string } }) {
  return <article>Showing {params.slug}</article>;
}

// Pages Router
export default function Page() {
  const { query } = useRouter();
  return <article>Showing {query.slug}</article>;
}

```

Q 7.08 Image Optimization (next/image)

`next/image` is a drop-in `<img>` replacement that handles modern formats (AVIF/WebP), responsive sizes, lazy loading, and CDN caching.

```

import Image from "next/image";

<Image
  src="/hero.jpg"
  alt="Hero"
  width={1200}
  height={600}
  priority // eagerly load (above-the-fold)
  sizes="(max-width: 768px) 100vw, 50vw"
/>

```

WHAT YOU GET

- On-demand resizing — original ships as smaller variants per device.
- Modern formats (AVIF/WebP) with fallback.
- Native lazy loading for off-screen images.
- Automatic prevention of CLS by reserving space.
- Configurable remote-image domains in `next.config.js`.

Q 7.09

Client-Side Routing (next/link)

`<Link>` from `next/link` performs client-side navigation: no full page reload, prefetches the destination's chunks when the link enters the viewport.

```
import Link from "next/link";

<Link href="/about">About</Link>
<Link href={{ pathname: "/blog/[slug]", query: { slug: "hello" } }}>Read</Link>

// Programmatic
import { useRouter } from "next/navigation"; // App Router
const router = useRouter();
router.push("/dashboard");
```

Server Components rendered above a `<Link>` are prefetched too — so navigation feels instant once the chunk arrives.

Q 7.10

React Server Components (App Router)

RSC lets some components render *only on the server* — they never ship to the client. The result is less JS shipped to the browser and direct access to backend resources from components.

Component type	Where it runs	Allowed
Server Component (default)	Server only	fetch DB / file system / secrets; no hooks, no event handlers
Client Component (<code>"use client"</code>)	Server + client	useState/useEffect, event handlers; no DB calls in component code

```
// Server component (no "use client")
import { db } from "@/lib/db";

export default async function Page() {
  const users = await db.user.findMany();
  return <UserList users={users} />; // RSC, no client JS
}

// Client component
"use client";
export function Counter() {
  const [n, set] = useState(0);
```

```
return <button onClick={() => set(n+1)}>{n}</button>;
}
```

Server Components stream HTML and serialized data to the client. Interactivity comes from Client Components further down the tree.

Q 7.11 SEO in Next.js

Next.js's server rendering means crawlers see fully rendered HTML — already a big SEO win over plain CSR. The framework also provides explicit metadata APIs.

APP ROUTER

```
// app/blog/[slug]/page.tsx
import type { Metadata } from "next";

export async function generateMetadata({ params }): Promise<Metadata> {
  const post = await getPost(params.slug);
  return {
    title: post.title,
    description: post.excerpt,
    openGraph: { images: [post.cover] },
  };
}
```

PAGES ROUTER

```
import Head from "next/head";

<Head>
  <title>{post.title}</title>
  <meta name="description" content={post.excerpt} />
</Head>
```

OTHER LEVERS

- `app/sitemap.ts` and `app/robots.ts` generate sitemap and robots.txt.
- Semantic HTML and good page structure.
- JSON-LD structured data via `<script type="application/ld+json">`.
- Fast Core Web Vitals (LCP, CLS, INP) — Next.js's image and font handling help.

Q 7.12 Route Protection (Auth)

Several layers, often combined:

MIDDLEWARE (EDGE)

```
// middleware.ts – runs before any route handler
import { NextResponse } from "next/server";

export function middleware(req) {
  const token = req.cookies.get("session")?.value;
  if (!token && req.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", req.url));
  }
  return NextResponse.next();
}

export const config = { matcher: ["/dashboard/:path*", "/admin/:path*"] };
```

SERVER COMPONENT / SERVER ACTION

```
export default async function Page() {
  const session = await getServerSession();
  if (!session) redirect("/login");
  return <Dashboard user={session.user} />;
}
```

CLIENT GUARD (UI ONLY)

```
"use client";
const { status } = useSession();
if (status === "unauthenticated") return <LoginPrompt />;
```

Treat client-side checks as UX, not security. Real protection lives in middleware (for redirects) and server-side checks in route handlers/server components (for data access).

Q 7.13 ISR — Incremental Static Regeneration

ISR gives you the speed of SSG with the freshness of SSR. Pages are statically generated, then re-generated in the background on a schedule or on demand.

TIME-BASED

```
// Pages Router
export async function getStaticProps() {
```

```

return {
  props: { data: await getData() },
  revalidate: 60,    // page becomes stale after 60s; next request triggers regen
};
}

// App Router
export const revalidate = 60;    // segment-level
const data = await fetch(url, { next: { revalidate: 60 } });

```

ON-DEMAND

```

// app/api/revalidate/route.ts
import { revalidatePath, revalidateTag } from "next/cache";

export async function POST(req) {
  revalidateTag("products");    // invalidate by tag
  revalidatePath("/products");  // invalidate a path
  return Response.json({ ok: true });
}

```

First user after invalidation still gets the stale page (instant); the regeneration happens in the background and serves fresh content to subsequent users.

Q 7.14 Middleware in Next.js

Middleware runs on the edge — before a request reaches your route handler. Common use cases: auth checks, redirects, locale detection, A/B test cookie injection, geolocation routing.

```

// middleware.ts at the project root
import { NextResponse, NextRequest } from "next/server";

export function middleware(req: NextRequest) {
  // Inject a cookie
  const res = NextResponse.next();
  if (!req.cookies.get("visitor-id")) {
    res.cookies.set("visitor-id", crypto.randomUUID());
  }

  // Redirect / rewrite
  const country = req.geo?.country ?? "US";
  if (country === "DE" && req.nextUrl.pathname === "/") {
    return NextResponse.rewrite(new URL("/de", req.url));
  }

  return res;
}

```

```
export const config = {  
  matcher: ["/(?!(?!_next/static|favicon).*)"], // skip static assets  
};
```

Middleware runs on the Edge runtime — a subset of Node APIs (no `fs`, limited Node built-ins). Heavy logic belongs in route handlers; middleware should be fast.

SECTION 08

Node.js

13 questions

Q 8.01

What is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine that runs JS outside the browser — primarily on servers. It pairs V8 with **libuv** (cross-platform async I/O) to provide a non-blocking, event-driven model.

WHY PEOPLE USE IT

- One language (JS/TS) for frontend and backend.
- Non-blocking I/O — handles many concurrent connections on one thread.
- Massive ecosystem (npm — the largest package registry).
- Great for I/O-bound services: APIs, real-time apps, streaming, serverless functions.

WHERE IT'S A BAD FIT

- CPU-bound work (heavy crypto, image/video processing) — blocks the event loop unless offloaded to Worker Threads or external services.

Q 8.02

How Node.js Works Under the Hood

Node's runtime has three main pieces working together:

- **V8** — Google's JS engine; parses and executes JavaScript.
- **libuv** — C library providing the event loop and a thread pool for I/O and other blocking work (file system, DNS).
- **Node bindings** — C++ glue exposing libuv and platform APIs to JS (fs, net, http, crypto, etc.).

LIFECYCLE OF AN I/O CALL

1. Your JS calls `fs.readFile(...)` with a callback.
2. Node dispatches the work to libuv's thread pool (or kernel async I/O for sockets).
3. The main thread keeps running other JS — non-blocking.
4. When the I/O completes, libuv queues the callback in the appropriate event-loop phase.
5. The event loop picks it up and runs the callback on the main JS thread.

Q 8.03

Why is Node.js Single-Threaded?

Node's JS execution runs on a *single main thread*. That's a deliberate design choice — it avoids the entire class of multithreading bugs (race conditions, deadlocks, locking overhead) and matches the most common workload: I/O-bound services that spend most of their time waiting on the network or disk.

IT'S NOT ONLY ONE THREAD

- libuv maintains a **thread pool** (default size 4) used for blocking ops like file I/O, DNS, and some crypto.
- The V8 engine has its own background threads for GC and compilation.
- Node ships **Worker Threads** for true CPU parallelism in JS.
- The **cluster** module forks multiple Node processes that share a port.

CPU-heavy work on the main thread blocks every request — common interview gotcha. Offload to Worker Threads, a child process, or an external worker.

Q 8.04 How Node Handles Concurrency

Node achieves concurrency through **async I/O** and the event loop, not threads. The runtime can have thousands of in-flight operations while the main thread runs just one JS function at a time.

THE MECHANISMS

- **Event loop** — schedules callbacks for completed async operations. Phases: timers → pending callbacks → poll → check → close.
- **libuv thread pool** — for ops that *are* blocking at the OS level (file I/O, some DNS, crypto).
- **OS-level async I/O** — for sockets (epoll on Linux, kqueue on BSD, IOCP on Windows).
- **Worker Threads** — for CPU-bound parallelism inside one Node process.
- **Cluster / child processes** — for scaling across CPU cores.

```
// CPU-heavy work — offload to a Worker
import { Worker } from "node:worker_threads";

const worker = new Worker("./heavy.js");
worker.postMessage(input);
worker.on("message", result => { /* ... */ });
```

Q 8.05 Modules in Node.js (CommonJS vs ES Modules)

Aspect	CommonJS	ES Modules (ESM)
Import	<code>const x = require("x")</code>	<code>import x from "x"</code>
Export	<code>module.exports = ...</code>	<code>export ...</code>
Loading	Synchronous	Asynchronous

Aspect	CommonJS	ES Modules (ESM)
Resolution	Runtime	At parse / build time
File ext	<code>.js</code> (default)	<code>.mjs</code> or <code>"type": "module"</code> in package.json
Top-level await	No	Yes

```
// CommonJS
const fs = require("fs");
module.exports = { hello };

// ESM
import fs from "node:fs/promises";
export function hello() {}
```

ESM is the modern default for new Node projects. Interop is mostly seamless: ESM can import CommonJS; CommonJS can `require()` ESM since Node 22 (with caveats). Note the `node:` prefix for built-ins — recommended in modern code.

Q 8.06

V8 Engine

V8 is the open-source JavaScript engine developed by Google for Chrome. Node.js embeds it to execute JavaScript on the server.

WHAT V8 DOES

- **Parse** JS source into an AST.
- **Compile** via a tiered system: *Ignition* (bytecode interpreter) for fast startup, then *TurboFan* for optimizing hot code into machine code based on profiling.
- **Execute** the resulting code on the host CPU.
- **Manage memory** with a generational, mark-and-sweep garbage collector.
- **Optimize and de-optimize** based on runtime behavior (hidden classes, inline caches).

Common V8 gotchas: monomorphic property access is faster than polymorphic; `delete` deoptimizes objects; very large flat arrays beat huge sparse objects.

Q 8.07

Control Flow in Node.js

Control flow refers to the order operations execute. In Node, most non-trivial flow is async — you compose callbacks, promises, or `async` / `await`.

THREE ERAS

- **Callbacks (Node-style)** — error first: `(err, result) => ...`. Powerful but nests badly.
- **Promises** — composable, with `.then / .catch`; combinators (`Promise.all`, etc.).
- **async/await** — synchronous-looking syntax over promises. Default in modern Node.

```
// Sequential
const user = await getUser(id);
const orders = await getOrders(user.id);

// Parallel
const [user, prefs] = await Promise.all([getUser(id), getPrefs(id)]);

// Loop with concurrency control (p-limit)
import pLimit from "p-limit";
const limit = pLimit(5);
await Promise.all(items.map(it => limit(() => process(it))));
```

A common bug: `forEach` doesn't await async callbacks. Use a `for...of` loop for sequential, or `Promise.all(items.map(...))` for parallel.

Q 8.08 CORS — Cross-Origin Resource Sharing

Browsers enforce the **Same-Origin Policy**: JS on origin A can't read responses from origin B by default. **CORS** is the server's way to opt in.

THE MECHANISM

1. Browser sends the request with an `Origin` header.
2. For "non-simple" requests (custom headers, methods other than GET/POST/HEAD, JSON content type) the browser first sends an `OPTIONS` **preflight**.
3. The server must respond with `Access-Control-Allow-Origin` (and friends) to authorize the actual request.

```
// Express
import cors from "cors";
app.use(cors({
  origin: ["https://app.example.com"],
  credentials: true, // allow cookies
  methods: ["GET", "POST"],
  allowedHeaders: ["Content-Type", "Authorization"],
}));
```

COMMON PITFALLS

- Setting `Access-Control-Allow-Origin: *` with `credentials: true` — browsers reject it. Echo the request's origin instead.
- Forgetting to handle the OPTIONS preflight at every relevant route.
- Confusing CORS errors with server errors — the request actually *reached* the server; the browser is blocking the response.

Q 8.09 Streams and Piping

A **stream** is an abstraction for reading from or writing to data in chunks instead of loading it all in memory. **Piping** connects a readable stream's output to a writable stream's input.

STREAM TYPES

- **Readable** — sources: `fs.createReadStream`, HTTP request body, `process.stdin`.
- **Writable** — sinks: `fs.createWriteStream`, HTTP response, `process.stdout`.
- **Duplex** — both directions on the same stream (TCP sockets).
- **Transform** — duplex that modifies data passing through (gzip, encryption).

```
import { createReadStream, createWriteStream } from "node:fs";
import { createGzip } from "node:zlib";
import { pipeline } from "node:stream/promises";

// Compress a big file without loading it into memory
await pipeline(
  createReadStream("big.log"),
  createGzip(),
  createWriteStream("big.log.gz"),
);
```

`pipeline()` handles backpressure and error propagation properly. The older `readable.pipe(writable)` API leaks resources when errors happen.

Q 8.10 The Cluster Module

Node runs JS on a single thread, so a single process uses one CPU core. The **cluster** module forks N worker processes that share the same server port, letting one Node app use all cores.

```
import cluster from "node:cluster";
import os from "node:os";
import http from "node:http";
```

```

if (cluster.isPrimary) {
  for (let i = 0; i < os.cpus().length; i++) cluster.fork();
  cluster.on("exit", (worker) => {
    console.log("worker died, restarting");
    cluster.fork(); // self-healing
  });
} else {
  http.createServer((req, res) => res.end(`pid ${process.pid}`)).listen(3000);
}

```

In production, most teams use a process manager (PM2, systemd) or a container orchestrator (Kubernetes) instead of `cluster` directly — they handle restarts, zero-downtime reloads, and resource limits.

Q 8.11 WebSockets in Node.js

WebSockets provide a bidirectional, persistent connection between client and server over a single TCP connection. They start as an HTTP request with an `Upgrade` header, then switch protocols.

SERVER WITH WS

```

import { WebSocketServer } from "ws";
import http from "node:http";

const server = http.createServer();
const wss = new WebSocketServer({ server });

wss.on("connection", (socket) => {
  socket.on("message", (data) => {
    // Echo to all connected clients
    for (const client of wss.clients) {
      if (client.readyState === client.OPEN) client.send(data);
    }
  });
});

server.listen(3000);

```

WHEN TO USE WHAT

- **WebSockets** — full-duplex, low latency, real-time apps (chat, presence, multiplayer games, collaborative editing).
- **Server-Sent Events (SSE)** — simpler one-way server-to-client streaming over plain HTTP; great for notifications and live dashboards.
- **Long polling** — fallback when sockets aren't an option; high overhead.

Production WebSocket setups usually add a Redis pub/sub (or NATS) layer so multiple Node instances can fan messages out across a cluster.

Q 8.12 package.json

`package.json` is the manifest of a Node project. It declares metadata, dependencies, scripts, and how the package is consumed.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "type": "module",                // use ESM by default
  "private": true,                 // prevents accidental publish
  "engines": { "node": ">=20.0.0" },
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "vitest",
    "lint": "eslint ."
  },
  "dependencies": {
    "react": "^19.0.0"             // runtime — installed in production
  },
  "devDependencies": {
    "typescript": "^5.5.0",        // build/test — not in prod bundles
    "vitest": "^2.0.0"
  },
  "peerDependencies": {
    "react": ">=18"               // for libraries — host app provides
  }
}
```

VERSION RANGE SYNTAX

- `^1.2.3` — any 1.x.y with $x \geq 2$ (most common; updates within a major).
- `~1.2.3` — any 1.2.x (patch only).
- `1.2.3` — exact pin.
- `*` or `latest` — anything.

A lockfile (`package-lock.json` , `pnpm-lock.yaml` , `yarn.lock`) pins the actual installed versions for reproducible builds. Commit it.

Q 8.13

npm — Node Package Manager

npm is the default package manager for Node, plus the registry at [npmjs.com](https://www.npmjs.com) — the world's largest software package registry.

COMMON COMMANDS

- `npm init -y` — create a `package.json` with defaults.
- `npm install pkg` — install and add to `dependencies`.
- `npm install -D pkg` — install as a dev dependency.
- `npm uninstall pkg` — remove.
- `npm update` — bump within ranges.
- `npm outdated` — show packages with newer versions available.
- `npm audit` / `npm audit fix` — check / patch known vulnerabilities.
- `npm run script-name` — run a script from `package.json`.
- `npx pkg` — run a package's binary without installing it globally.
- `npm ci` — clean install matching the lockfile exactly (used in CI).

NPM VS PNPM VS YARN

	npm	pnpm	yarn
Default with Node	Yes	No	No
Disk usage	Higher (duplicates)	Lowest (content-addressable store)	Medium
Workspaces	Yes	Yes (best for monorepos)	Yes
Speed	Improved a lot since v7	Fastest	Fast

SECTION 09

Backend & APIs

14 questions

Q 9.01

RESTful APIs

REST (Representational State Transfer) is an architectural style for designing networked APIs around **resources** identified by URLs, manipulated with standard HTTP methods.

| CONSTRAINTS

- **Client-server** — separation of concerns.
- **Stateless** — every request carries everything needed to process it.
- **Cacheable** — responses declare cacheability.
- **Uniform interface** — resources via URLs, standard HTTP methods and status codes.
- **Layered system** — proxies, gateways, CDNs are transparent.

| METHOD CONVENTIONS

Method	Action	Idempotent
GET /users	List	Yes
GET /users/:id	Read one	Yes
POST /users	Create	No
PUT /users/:id	Replace	Yes
PATCH /users/:id	Partial update	No (often)
DELETE /users/:id	Remove	Yes

Q 9.02

GraphQL vs REST

Aspect	REST	GraphQL
Endpoints	Many — one per resource	One — typically <code>/graphql</code>
Data shape	Defined by the server	Defined by the client (query)
Over/under-fetching	Common — fixed payloads	Rare — ask for exactly what you need
Versioning	URL or header (v1, v2)	Evolutionary — deprecate fields
Caching	HTTP caching is straightforward	Needs library-level cache (Apollo, Relay, urql)
Best for		

Aspect	REST	GraphQL
	Public APIs, simple CRUD, file uploads	Complex relational data, mobile apps where bandwidth matters

```
# GraphQL query – only what the UI needs
query {
  user(id: "1") {
    name
    posts(limit: 5) {
      title
      comments { author }
    }
  }
}
```

Trade-off: GraphQL gives the client flexibility at the cost of more server complexity (N+1 queries, query complexity limits, harder caching). REST is simpler and benefits from the whole HTTP toolchain.

Q 9.03

SOLID Principles

SOLID is five OOP principles for maintainable, extensible code.

	Principle	Idea
S	Single Responsibility	A class should have one reason to change.
O	Open/Closed	Open for extension, closed for modification.
L	Liskov Substitution	Subtypes must be usable wherever the parent type is, without surprises.
I	Interface Segregation	Many small, specific interfaces beat one fat one.
D	Dependency Inversion	Depend on abstractions, not concretions.

QUICK EXAMPLES

- **SRP violation** — a `User` class that also sends emails. Split out an `EmailService`.
- **OCP win** — strategy pattern: add a new payment provider by adding a class, not modifying existing code.
- **LSP violation** — `Square extends Rectangle` where setting width also sets height. Subtle bugs.
- **ISP** — instead of `IUserService` with 20 methods, split into `IUserReader`, `IUserWriter`, `IUserAuth`.
- **DIP** — service constructor takes an `ILogger` interface, not a concrete `FileLogger`.

Q 9.04

Microservices

An architectural style where a system is built as a suite of small, independently deployable services that communicate over the network (HTTP/REST, gRPC, message queues).

| CHARACTERISTICS

- Each service owns one business capability and its own database.
- Services are loosely coupled and independently deployable.
- Polyglot — different services can use different languages and storage tech.
- Communicate via well-defined APIs or async messaging.

| TRADE-OFFS

Pros	Cons
Scale teams independently	Distributed-systems complexity (latency, partial failure, ordering)
Tech freedom per service	Need for observability (tracing, centralized logging)
Fault isolation	Data consistency across services (sagas, outbox pattern)
Independent deploys	Operational overhead — orchestration, CI/CD per service

Start with a monolith. Split into services only when team size, scale, or independent deployment requirements actually demand it. The "majestic monolith" is still the right answer for most teams.

Q 9.05

Payload, Request, and Response

Payload is the actual data carried in a message — the body of an HTTP request or response. Distinct from headers and the URL.

| TYPICAL HTTP REQUEST

```
POST /api/users HTTP/1.1
Host: api.example.com
Content-Type: application/json
Authorization: Bearer eyJ...
Content-Length: 47

{ "name": "Dincho", "email": "x@example.com" }  <-- payload
```

TYPICAL HTTP RESPONSE

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/users/42

{ "id": 42, "name": "Dincho", "createdAt": "2026-06-08T..." }
```

Payloads can be JSON (most common), form-encoded data, multipart (file upload), binary, XML, Protobuf, etc.

Q 9.06

Anatomy of an HTTP Request

Every HTTP request has the same structure: start-line, headers, optional body.

```
POST /api/login HTTP/1.1          <-- method, path, version
Host: api.example.com             <-- headers...
User-Agent: Mozilla/5.0
Content-Type: application/json
Accept: application/json
Authorization: Bearer xyz...
Content-Length: 53

{"email":"x@example.com","password":"hunter2"} <-- body
```

Part	Purpose
Method	What action to perform (GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD)
Path + query	Resource to act on, with optional filters
Headers	Metadata: content type, auth, caching, host, accepted formats
Body	The payload — omitted for GET/HEAD

STATUS CODE CATEGORIES

- **1xx** — informational (rarely seen)
- **2xx** — success (200 OK, 201 Created, 204 No Content)
- **3xx** — redirect (301 permanent, 302 found, 304 not modified)
- **4xx** — client error (400, 401, 403, 404, 409, 422, 429)
- **5xx** — server error (500, 502, 503, 504)

Q 9.07

REST API Caching

HTTP has a rich cache model that REST APIs can leverage. Caching can happen in the client, in a CDN, in a reverse proxy, or in the server.

RESPONSE HEADERS THAT CONTROL IT

- `Cache-Control: public, max-age=3600` — cacheable by anyone for 1 hour.
- `Cache-Control: private, no-cache` — only the browser caches; must revalidate.
- `Cache-Control: no-store` — never cache (for sensitive data).
- `ETag: "abc123"` — opaque content fingerprint.
- `Last-Modified: ...` — timestamp of last change.
- `Vary: Accept-Encoding, Authorization` — cache key depends on these request headers.

CONDITIONAL REQUESTS

```
GET /api/users/42 HTTP/1.1
If-None-Match: "abc123"
```

```
HTTP/1.1 304 Not Modified # No body sent – client uses cached copy
```

Server-side caches (Redis, Memcached) handle data the database is slow to produce. CDN edge caches (Cloudflare, Fastly) handle public, geographically distributed traffic. Pair both for hot reads.

Q 9.08

REST API Best Practices

- **Nouns, not verbs, in URLs** — `POST /orders`, not `POST /createOrder`.
- **Plural resource names** — `/users`, `/users/:id`.
- **Use proper status codes** — 201 Created with a `Location` header, 204 No Content for successful empty responses, 422 for validation errors.
- **Version the API** — `/v1/...` or a header. Plan for breaking changes.
- **Filter, sort, paginate via query params** — `?status=open&sort=-createdAt&page=2&limit=20`. Use cursor pagination for large datasets.
- **Consistent JSON shape** — predictable field names, dates as ISO 8601 UTC.
- **Errors with structure** — `{ error: { code, message, details } }`. Don't leak stack traces.
- **Idempotency keys** on POST for unsafe operations that may be retried (payments).
- **Auth via Bearer tokens or OAuth** — never roll your own.
- **Rate limit** — return 429 with `Retry-After`.
- **Document with OpenAPI** — generate clients and request validators from one source of truth.

- **HATEOAS** (links in responses) — purist REST; in practice, most APIs skip it.

Q 9.09

SQL Injection

SQL injection happens when user input is concatenated into a SQL query, allowing attackers to alter the query's logic. One of the oldest and most damaging classes of vulnerability.

```
// VULNERABLE – string concatenation
const q = `SELECT * FROM users WHERE email = '${email}'`;
db.query(q);
// If email is: ' OR '1'='1
// Query becomes: SELECT * FROM users WHERE email = '' OR '1'='1' -- returns all rows
```

DEFENSES

- **Parameterized queries / prepared statements** — the canonical fix. Values are sent separately from the SQL.
- **ORM / query builder** — Prisma, TypeORM, Knex, Drizzle all parameterize by default.
- **Allow-list inputs** for things that can't be parameterized (column names, ORDER BY direction).
- **Least privilege** — the DB user the app connects as should not be able to **DROP** tables.
- **Validate input shape** — but never as the only defense.

```
// SAFE – parameterized
db.query("SELECT * FROM users WHERE email = $1", [email]);

// SAFE – Prisma
const u = await prisma.user.findUnique({ where: { email } });
```

Q 9.10

HTTP Caching

HTTP defines a layered cache hierarchy: browser cache → corporate proxy → CDN edge → reverse proxy → origin. Each layer respects the same cache headers.

TWO STRATEGIES

Strategy	How	Use for
Freshness	<code>Cache-Control: max-age=N</code>	Static assets, public data that rarely changes
Validation	<code>ETag + If-None-Match</code> , or <code>Last-Modified + If-Modified-Since</code>	Data that might change but is expensive to regenerate

ASSET FINGERPRINTING

Bundlers emit filenames like `app.7d4f2.js`. The hash changes when content does, so you can set `Cache-Control: public, max-age=31536000, immutable` safely — a new deploy generates a new filename.

HTML CACHING

Usually `Cache-Control: no-cache` (must revalidate) for the HTML entry point, so users get the latest references to fingerprinted assets right away.

Q 9.11 Scaling a SQL Database

VERTICAL (SCALE UP)

Bigger machine — more CPU, RAM, faster disks. Easy and effective up to a point; ceiling is hardware limits and price/performance.

HORIZONTAL (SCALE OUT)

- **Read replicas** — primary handles writes, replicas serve reads. Watch for replication lag.
- **Sharding** — partition data across multiple primaries by some key (user ID, region). Complex to operate; queries that cross shards are slow.
- **Connection pooling** — PgBouncer/RDS Proxy to handle thousands of clients with a small backend pool.

REDUCE LOAD ON THE DB

- **Cache hot reads** — Redis/Memcached in front of the DB.
- **Materialized views** for expensive aggregates.
- **Index strategically** — but be aware indexes slow writes and use memory.
- **Move time-series and analytics** to purpose-built stores (ClickHouse, BigQuery).
- **Async processing** via queues for non-critical writes.

Modern managed offerings (AWS Aurora, Google AlloyDB, Neon, PlanetScale) hide a lot of this complexity behind autoscaling and built-in pooling.

Q 9.12 Deployment Strategies

Strategy	How it works	Risk
Recreate	Stop old, deploy new.	Downtime — simple, smallest infra.

Strategy	How it works	Risk
Rolling	Replace instances a few at a time.	Mixed-version traffic during rollout.
Blue/Green	Two identical environments; switch traffic atomically.	Costs 2× during cutover; fast rollback.
Canary	Send a small % of traffic to the new version; expand if healthy.	Need good metrics and easy rollback.
Feature flags	Deploy "dark"; toggle on for some users.	Decouples deploy from release; flag debt to manage.
Shadow	Mirror traffic to the new version without affecting users.	Used to validate behavior under load.

In practice, the modern stack is rolling deploys via Kubernetes/ECS, with feature flags for risky changes and a canary stage for the riskiest. Most teams don't need full blue/green.

Q 9.13 JWT — JSON Web Token

A **JWT** is a self-contained token used for stateless authentication. It has three Base64URL-encoded parts joined with dots: `header.payload.signature`.

```
// Header
{ "alg": "HS256", "typ": "JWT" }

// Payload (claims)
{ "sub": "user_42", "name": "Dincho", "exp": 1735689600, "iat": 1735603200 }

// Signature
HMAC_SHA256(base64(header) + "." + base64(payload), secret)
```

HOW IT FLOWS

1. User logs in → server signs a JWT and returns it.
2. Client stores it (cookie or memory; **not** localStorage for sensitive data).
3. Client sends it with each request: `Authorization: Bearer eyJ...`
4. Server verifies the signature and reads claims — no DB lookup needed.

PITFALLS

- **Can't be revoked easily** — once issued, it's valid until expiry. Use short-lived access tokens + refresh tokens, or maintain a revocation list.
- **Sensitive data in the payload** — it's *signed*, not encrypted. Anyone can decode it.

- **"None" algorithm bug** — always reject `alg: none`; libraries usually do, but verify your config.
- **Storage** — HTTP-only Secure cookies are safer than localStorage (which is JS-accessible and vulnerable to XSS).

Q 9.14**SQL vs NoSQL Databases**

Aspect	SQL (relational)	NoSQL
Data model	Tables with fixed schema	Documents, key-value, wide-column, graph
Schema	Strict	Flexible — schema-on-read
Joins	First-class	Limited or app-side
Consistency	ACID by default	Often eventual; varies by product
Scaling	Vertical, read replicas, sharding (harder)	Designed to scale horizontally
Examples	PostgreSQL, MySQL, SQL Server, Oracle	MongoDB (docs), Redis (KV), Cassandra (wide), Neo4j (graph), DynamoDB (KV/doc)
Best for	Structured data with relationships, transactions, reporting	Flexible schemas, massive scale, simple access patterns

HONEST ADVICE

For most apps, **start with PostgreSQL**. It handles JSON nicely, scales further than people think, and gives you ACID transactions for free. Reach for NoSQL when you have a specific access pattern that a relational DB serves badly (massive write throughput, graph traversals, KV at scale).